

x86 Assembly Reference for OSED

EXP-301 Operational Reference – Windows x86 User-Mode

OSED-Toolkit

2026-07-18

Contents

1	1. CPU Execution Model	5
1.1	Registers, Memory, and Instructions	5
1.2	Sub-Register Aliasing	5
1.3	Little-Endian Byte Order	6
1.4	Value vs. Address vs. Contents	6
2	2. Register Reference	6
2.1	General-Purpose Registers	6
2.2	Segment Registers	7
2.3	Implicit Register Usage Summary	7
3	3. Memory Operands and Pointers	8
3.1	Intel Operand Syntax	8
3.2	MOV vs. LEA	8
3.3	Effective Address Components	8
3.4	Size Qualifiers	8
3.5	Pointer Chains	9
3.6	Arrays	9
3.7	Structures	9
3.8	Memory Permissions	9
4	4. Endianness	9
4.1	Byte Order in Memory	9
4.2	Impact on Exploit Development	10
5	5. EFLAGS and Conditional Execution	10
5.1	Flags Relevant to Reverse Engineering	10
5.2	CMP and TEST	11
5.3	Signed vs. Unsigned Conditional Jumps	11
5.4	Why This Matters	12
5.5	SETcc and CMOVcc	12
6	6. Core Instruction Reference	12
6.1	Data Movement	12
6.2	Arithmetic	14
6.3	Logic and Bit Manipulation	15
6.4	Control Flow	16
6.5	String and Memory Operations	17
6.6	Other Commonly Encountered	18
7	7. Stack Fundamentals	19
7.1	Growth Direction	19

7.2	Stack Reserve vs. Stack Commit	19
7.3	State Transitions	19
7.4	Guard Pages	20
8	8. Function Stack Frames	21
8.1	Standard Prologue	21
8.2	Standard Epilogue	21
8.3	Frame Layout	21
8.4	Frame-Pointer Omission (FPO)	21
8.5	Inlined Functions	21
8.6	Tail Calls	21
9	9. Calling Conventions	22
9.1	cdecl (C Declaration)	22
9.2	stdcall (Standard Call)	22
9.3	fastcall	22
9.4	thiscall (MSVC C++)	22
9.5	On Entry to the Callee	23
10	10. Normal CALL vs. RET-Based Invocation	23
10.1	Normal CALL	23
10.2	RET-Based Invocation (ROP)	23
11	11. Buffers and Stack Overflows	24
11.1	Stack Buffer Layout	24
11.2	Overflow Mechanics	24
11.3	Offset Calculation	24
11.4	Cyclic Pattern Verification	25
11.5	Bad Characters	25
11.6	Partial Overwrites	25
11.7	Stack Pivots	25
11.8	SEH Overwrites	25
12	12. Arithmetic Used in Exploit Development	25
12.1	Offset Arithmetic	25
12.2	Two's Complement	26
12.3	Null-Byte Avoidance via Negative Arithmetic	26
12.4	Alignment	26
12.5	Integer Truncation and Extension	26
12.6	Wraparound	26
13	13. ROP Gadget Semantics	26
13.1	Register Load	27
13.2	Register Exchange	27
13.3	Dereference (Memory Read)	27
13.4	Memory Write	27
13.5	Arithmetic	27
13.6	Stack Adjustment	27
13.7	Stack Pivot	28
13.8	PUSHAD in ROP	28
13.9	Conditional Gadgets	28
13.10	Side Effects	28
14	14. PUSHAD-Based API Calls	28
14.1	PUSHAD Push Order and Memory Layout	28
14.2	Using PUSHAD to Build a Call Frame	29
15	15. DEP and Page Protections	29

15.1 Executable vs. Non-Executable Memory	29
15.2 Why Stack Shellcode Faults	29
15.3 VirtualProtect	29
15.4 VirtualAlloc	30
15.5 Protection Constants	30
15.6 Allocation Type Constants	30
15.7 WinDbg Verification	30
16 16. ASLR and Module Addressing	30
16.1 Image Base and RVA	30
16.2 ASLR	31
16.3 Fixed vs. Randomized Modules	31
16.4 Exploiting Non-ASLR Modules	31
16.5 Partial Pointer Overwrites	31
16.6 Information Leaks	31
17 17. Windows Process Structures and PEB Walking	31
17.1 The Path	31
17.2 TEB (Thread Environment Block)	32
17.3 PEB (Process Environment Block)	32
17.4 PEB_LDR_DATA	32
17.5 _LDR_DATA_TABLE_ENTRY	32
17.6 List Entry Pointers and Structure Offsets	32
17.7 Assembly Walkthrough	33
17.8 Module Order (Windows 7/10 32-bit, typical)	33
17.9 WinDbg Verification	33
18 18. PE Structures and Export Resolution	33
18.1 Header Chain	33
18.2 IMAGE_EXPORT_DIRECTORY	34
18.3 Resolution Algorithm	34
18.4 Assembly Implementation	34
18.5 Forwarded Exports	35
18.6 API Sets (Conceptual)	35
19 19. Position-Independent Shellcode	35
19.1 Core Constraints	35
19.2 Self-Location: JMP-CALL-POP	35
19.3 Stack Strings	35
19.4 Null-Byte Avoidance	35
19.5 Direction Flag	36
19.6 Register Preservation	36
19.7 Stack Safety	36
20 20. API Hashing	36
20.1 Why Hash?	36
20.2 Common Hash Algorithm: ROR-13-ADD	36
20.3 Case Sensitivity	37
20.4 Collision Handling	37
20.5 Module + Function Hashing	37
20.6 Finding Hash Constants	37
21 21. String Construction in Shellcode	37
21.1 Pushing Strings Backward	37
21.2 How to Compute the DWORD Values	38
21.3 Handling Odd Lengths	38
21.4 Avoiding Bad Characters in Strings	38
21.5 Unicode (UTF-16LE) Strings	38

22	22. Format-String Assembly Concepts	38
22.1	Variadic Arguments on x86	38
22.2	Format Specifiers	39
22.3	Why Format Strings Are Dangerous	39
22.4	Parameter Indexing	39
22.5	Write-What-Where with %n	39
22.6	Partial Writes with %hn	39
22.7	%hhn for Byte-Granularity Writes	39
22.8	Little-Endian Address Placement	40
23	23. IDA Pro Reading Guide	40
23.1	Views	40
23.2	Stack Variables	40
23.3	Argument Labels	40
23.4	Cross-References (Xrefs)	40
23.5	Imported Functions	40
23.6	Function Boundaries	40
23.7	Switch Tables	40
23.8	Thunk Functions	41
23.9	Type Propagation	41
23.10	Debugger vs. Disassembler Disagreement	41
24	24. WinDbg Reading Guide	41
24.1	Register Commands	41
24.2	Disassembly	41
24.3	Memory Display	41
24.4	Stack Commands	41
24.5	Module Commands	42
24.6	Memory Protection	42
24.7	Structure Display	42
24.8	Exception State	42
24.9	First-Chance vs. Second-Chance Exceptions	42
24.10	Context Distinction	42
24.11	Breakpoints	42
24.12	Exception Handling Configuration	43
24.13	Useful Expressions	43
25	25. Python Exploit-Development Equivalents	43
25.1	Packing and Unpacking	43
25.2	Byte Patterns	43
25.3	ROP Chain Construction	43
25.4	Bad Character Checking	44
25.5	Socket Communication	44
25.6	Address Formatting	44
25.7	Offset Calculation	44
25.8	ROR-13 Hash Computation	44
26	26. Common Analyst Mistakes	45
27	27. Fast-Reference Tables	45
27.1	Register Summary	45
27.2	Flags Summary	46
27.3	Operand Sizes	46
27.4	Conditional Jump Quick Reference	46
27.5	Calling Conventions	46
27.6	Stack Effects	47
27.7	Common Gadget Effects	47
27.8	Page Protection Constants	47

27.9 Allocation Type Constants	47
27.10PE Export Resolution Offsets	47
27.11PEB Walking Offsets	48
27.12SEH Chain Offsets	48
27.13Common WinDbg Commands	48
27.14Python Packing Formats	49
28 28. Worked State-Transition Exercises	49
28.1 Exercise 1: POP and XCHG	49
28.2 Exercise 2: CALL and RET	50
28.3 Exercise 3: RET N	50
28.4 Exercise 4: PUSH and POP Sequence	50
28.5 Exercise 5: LEAVE	51
28.6 Exercise 6: Pointer Dereference Chain	51
28.7 Exercise 7: LEA vs. MOV	52
28.8 Exercise 8: Stack Pivot	52
28.9 Exercise 9: VirtualProtect ROP Frame	53
28.10Exercise 10: PUSHAD	54
28.11Exercise 11: PEB Walking	54

1 1. CPU Execution Model

1.1 Registers, Memory, and Instructions

The x86 CPU operates on **registers** (fast named storage inside the CPU) and **memory** (byte-addressable RAM). Instructions read state, transform it, and write results back. The CPU fetches the next instruction from the address in **EIP**, decodes it, executes it, then advances EIP to the next instruction (unless the instruction itself changes EIP).

Key registers for OS/2 work:

Register	Width	Role
EAX	32	Accumulator; return values; implicit operand for mul/div
EBX	32	General purpose; base register in some addressing
ECX	32	Counter for loops and string ops; arg1 in fastcall
EDX	32	Data; high half of mul/div results; arg2 in fastcall
ESI	32	Source index for string ops
EDI	32	Destination index for string ops
ESP	32	Stack pointer – always points to the top of the stack
EBP	32	Frame pointer (by convention, not by hardware)
EIP	32	Instruction pointer – address of the next instruction
EFLAGS	32	Status and control flags (ZF, CF, SF, OF, PF, DF, etc.)

1.2 Sub-Register Aliasing

Each 32-bit general-purpose register has 16-bit and 8-bit aliases that refer to the same physical storage:

EAX (bits 31..0)
AX (bits 15..0)

AH (bits 15..8) AL (bits 7..0)

Writing AL changes bits 7..0 of EAX without clearing bits 31..8. Writing AX changes bits 15..0 without clearing bits 31..16. This matters when shellcode uses `mov al, 0x30` to avoid null bytes – the upper 24 bits of EAX retain whatever value they held.

The same pattern applies to EBX/BX/BH/BL, ECX/CX/CH/CL, EDX/DX/DH/DL. ESI, EDI, ESP, and EBP have 16-bit aliases (SI, DI, SP, BP) but no 8-bit aliases on x86-32.

1.3 Little-Endian Byte Order

x86 stores multi-byte values with the least significant byte at the lowest address:

Value: 0x41424344

Address: 0x0012FF80

Memory layout:

0x0012FF80: 44 (LSB)

0x0012FF81: 43

0x0012FF82: 42

0x0012FF83: 41 (MSB)

1.4 Value vs. Address vs. Contents

These are three different things:

EAX = 0x41424344

EAX holds the integer value 0x41424344.

EAX = 0x0012FF80

[0x0012FF80] = 0x41424344

EAX holds the address 0x0012FF80. The memory at that address contains 0x41424344. EAX is a pointer.

The bracket notation `[expr]` means “the value stored in memory at address `expr`”:

Expression	Meaning
EAX	The value in register EAX
[EAX]	The DWORD in memory at the address held in EAX
[EAX+4]	The DWORD in memory at address EAX+4
DWORD PTR [EAX+4]	Same, with explicit size qualifier

2. Register Reference

2.1 General-Purpose Registers

2.1.1 EAX – Accumulator

Return value for functions (all x86 calling conventions). Implicit destination for `mul`, `imul` (one-operand form), `div`, `idiv`, `cdq`, `cwde`, `lodsrb/d`, `xlat`. Commonly used as a scratch register by compilers.

2.1.2 EBX – Base

No implicit use in most instructions (except `xlat`, which reads `[EBX+AL]`). Callee-saved in all Windows calling conventions. Often used to hold a base address (module base, structure pointer) across function calls.

2.1.3 ECX – Counter

Implicit count for `loop`, `rep` prefix, `shl/shr/rol/ror` (CL form). First integer argument in `__fastcall`. this pointer in `__thiscall`. Caller-saved.

2.1.4 EDX – Data

High 32 bits of the 64-bit result from `mul/imul` (one-operand). High 32 bits of the 64-bit dividend for `div/ldiv`. Receives sign extension from `cdq` (EAX sign-extended into EDX:EAX). Second argument in `__fastcall`. Caller-saved.

2.1.5 ESI – Source Index

Source pointer for string instructions (`movsb/d`, `lodsb/d`, `cmps/d`). Callee-saved. Auto-incremented or decremented after string ops depending on DF.

2.1.6 EDI – Destination Index

Destination pointer for string instructions (`movsb/d`, `stosb/d`, `cmprsb/d`, `scasb/d`). Callee-saved. Auto-incremented or decremented after string ops depending on DF.

2.1.7 ESP – Stack Pointer

Points to the top of the stack (the most recently pushed value). Modified implicitly by `push`, `pop`, `call`, `ret`, `enter`, `leave`, `pushad`, `popad`, `int`. Manipulated explicitly by `sub esp, N` / `add esp, N` for frame allocation and cleanup.

2.1.8 EBP – Frame Pointer (by convention)

In framed functions, holds a snapshot of ESP taken during the prologue. Arguments are at positive offsets from EBP; locals at negative offsets. Not all functions use EBP as a frame pointer – optimized code with frame-pointer omission (FPO) uses EBP as a general register and references locals via ESP-relative offsets.

2.1.9 EIP – Instruction Pointer

Cannot be read or written directly by `mov`. Changed by `call`, `ret`, `jmp`, `jcc`, `int`, and exceptions. When you “overwrite EIP” in an exploit, you overwrite the saved return address on the stack; `ret` then loads that value into EIP.

2.1.10 EFLAGS

Status flags set by arithmetic and comparison instructions, read by conditional jumps and `setcc/cmovcc`. See Section 5 for details.

2.2 Segment Registers

Register	OSED relevance
CS	Code segment; defines current privilege level (ring 3 in user mode)
DS	Default data segment; implicit for most memory references
SS	Stack segment; implicit for ESP/EBP-based references
ES	Extra segment; implicit destination for string ops (EDI)
FS	Points to the Thread Environment Block (TEB); FS:[0x30] = PEB pointer
GS	Unused in 32-bit Windows user mode (used in x64 for TEB)

FS is the most operationally important segment register for OSED. The TEB (Thread Environment Block) is located at the base address of the FS segment. FS:[0x00] is the head of the SEH chain. FS:[0x30] is the pointer to the PEB (Process Environment Block).

2.3 Implicit Register Usage Summary

Instruction	Implicit registers
<code>mul reg</code>	EDX:EAX = EAX * reg
<code>imul reg</code>	EDX:EAX = EAX * reg (signed)

Instruction	Implicit registers
<code>div reg</code>	$EAX = EDX:EAX / \text{reg}$, $EDX = \text{remainder}$
<code>idiv reg</code>	$EAX = EDX:EAX / \text{reg}$ (signed), $EDX = \text{remainder}$
<code>cdq</code>	$EDX = \text{sign-extend}(EAX)$
<code>loop label</code>	$ECX = ECX - 1$; jump if $ECX \neq 0$
<code>rep movsb</code>	Copy ECX bytes from $[ESI]$ to $[EDI]$
<code>rep stosd</code>	Fill ECX dwords at $[EDI]$ with EAX
<code>lodsb</code>	$AL = [ESI]$; $ESI += 1$
<code>scasb</code>	Compare AL with $[EDI]$; $EDI += 1$
<code>call target</code>	push EIP_next ; $EIP = \text{target}$
<code>ret</code>	$EIP = [ESP]$; $ESP += 4$
<code>xlat</code>	$AL = [EBX + AL]$

3. Memory Operands and Pointers

3.1 Intel Operand Syntax

All examples use Intel syntax (destination, source):

```

mov eax, ebx           ; register to register: EAX = EBX
mov eax, [ebx]         ; memory to register: EAX = DWORD at address EBX
mov eax, [ebx+4]       ; EAX = DWORD at address EBX+4
mov eax, [ebx+ecx*4+8] ; EAX = DWORD at address EBX + ECX*4 + 8
lea eax, [ebx+ecx*4+8] ; EAX = EBX + ECX*4 + 8 (address calculation, no memory read)

```

3.2 MOV vs. LEA

This is one of the most commonly confused distinctions:

```

mov eax, [ebx+4] ; reads memory: EAX = the DWORD stored at address (EBX+4)
lea eax, [ebx+4] ; computes address: EAX = EBX + 4 (no memory access)

```

MOV with brackets dereferences – it goes to memory and fetches the value. LEA never touches memory – it computes the effective address and stores the result. Compilers use LEA as a fast multi-operand add/multiply:

```

lea eax, [ecx+ecx*2] ; EAX = ECX * 3
lea eax, [ecx*4+0x10] ; EAX = ECX * 4 + 16

```

3.3 Effective Address Components

The general form is `[base + index*scale + displacement]`:

Component	Allowed values
base	Any GPR
index	Any GPR except ESP
scale	1, 2, 4, or 8
displacement	Signed 8-bit or 32-bit immediate

The scale encodes element size: `*4` = dword array, `*2` = word array, `*1` (or omitted) = byte array.

3.4 Size Qualifiers

When the assembler cannot infer operand size from context, a PTR qualifier is required:

```

mov BYTE PTR [eax], 0      ; write 1 byte
mov WORD PTR [eax], 0      ; write 2 bytes
mov DWORD PTR [eax], 0     ; write 4 bytes
mov byte ptr [eax], cl     ; size inferred from CL (8-bit), PTR is documentation

```

Qualifier	Size	Bytes
BYTE PTR	8-bit	1
WORD PTR	16-bit	2
DWORD PTR	32-bit	4

3.5 Pointer Chains

Structures containing pointers to other structures create chains:

```

mov eax, [eax+0x0C]      ; follow pointer at offset 0x0C
mov eax, [eax+0x14]      ; follow pointer at offset 0x14 of the result
mov eax, [eax+0x08]      ; read value at offset 0x08

```

Each `mov reg, [reg+offset]` dereferences one level. This is the pattern used in PEB walking (Section 17).

3.6 Arrays

```

mov eax, [esi+ecx*4]      ; EAX = array[ecx], where elements are DWORDs

```

The scale factor `*4` encodes the element size. `ECX` is the index. `ESI` is the base address of the array.

3.7 Structures

Structure field access appears as a fixed displacement from a base register:

```

mov eax, [ebx+0x08]      ; read field at offset 0x08
mov [ebx+0x20], ecx      ; write field at offset 0x20

```

Multiple accesses to the same base with different fixed offsets indicate structure field access.

3.8 Memory Permissions

Not all memory is readable, writable, or executable. When the CPU attempts an operation that violates a page's protection, it raises an access violation:

- **Read from non-readable page** – access violation
- **Write to read-only page** – access violation
- **Execute from non-executable page** – access violation (DEP/NX)

Use `!address` or `!vprot` in WinDbg to check page protections.

4. Endianness

4.1 Byte Order in Memory

x86 is little-endian: the least significant byte is stored at the lowest address.

Value: 0x625011AF

Memory (low address first):

```

+0x00: AF
+0x01: 11
+0x02: 50

```

+0x03: 62

WinDbg `db` output: AF 11 50 62

WinDbg `dd` output: 625011AF

db shows raw bytes in memory order. dd reconstructs the DWORD by reading 4 bytes little-endian and displaying the integer value.

4.2 Impact on Exploit Development

4.2.1 Overwritten EIP

When you overwrite a saved return address with 0x625011AF, you write the bytes AF 11 50 62 into memory. The CPU reads these 4 bytes little-endian and loads 0x625011AF into EIP on ret.

4.2.2 ROP Chains

Every address in a ROP chain must be written in little-endian order. In Python:

```
import struct

addr = 0x625011AF
payload = struct.pack("<I", addr) # b'\xaf\x11Pb'
```

4.2.3 Python Packing

```
from struct import pack, unpack

# Pack a 32-bit integer as little-endian bytes
packed = pack("<I", 0x625011AF) # b'\xaf\x11Pb'

# Unpack 4 bytes back to an integer
value = unpack("<I", b'\xaf\x11Pb')[0] # 0x625011AF

# Common helper
def p32(v: int) -> bytes:
    return pack("<I", v & 0xFFFFFFFF)
```

4.2.4 Debugger Memory Display

0:000> dd esp L4

0012ff80 625011af 41414141 42424242 43434343

0:000> db esp L10

0012ff80 af 11 50 62 41 41 41 41-42 42 42 42 43 43 43 43

The dd view shows reconstructed DWORDs; the db view shows raw byte order. When constructing payloads, think in db order (the bytes you actually send) and verify with dd (the values the CPU will interpret).

4.2.5 Unicode and UTF-16

Windows uses UTF-16LE for Unicode strings. Each character is 2 bytes, little-endian. The ASCII character A (0x41) becomes 41 00 in UTF-16LE. This matters when comparing module names in PEB walking – the names are stored as UNICODE_STRING (UTF-16LE).

5 5. EFLAGS and Conditional Execution

5.1 Flags Relevant to Reverse Engineering

Flag	Bit	Set when
ZF (Zero)	6	Result is zero
CF (Carry)	0	Unsigned overflow/borrow
SF (Sign)	7	Result has MSB set (negative in signed interpretation)
OF (Overflow)	11	Signed overflow
PF (Parity)	2	Low byte of result has even number of 1-bits
DF (Direction)	10	String ops decrement (DF=1) or increment (DF=0) ESI/EDI

5.2 CMP and TEST

`cmp a, b` computes `a - b`, sets flags, discards the result.

`test a, b` computes `a AND b`, sets flags, discards the result. CF and OF are always cleared by `test`.

Common patterns:

```

cmp eax, 0      ; is EAX zero? (ZF=1 if yes)
test eax, eax   ; same check, shorter encoding, preferred by compilers
test al, al      ; same for 8-bit
cmp eax, ebx    ; compare two registers

```

5.3 Signed vs. Unsigned Conditional Jumps

The same flag state has different meanings depending on whether the comparison is signed or unsigned. The jump mnemonic tells you which interpretation applies:

5.3.1 Unsigned Comparisons (use CF and ZF)

Mnemonic	Condition	Flags
JA / JNBE	above (unsigned >)	CF=0 and ZF=0
JAE / JNB / JNC	above or equal	CF=0
JB / JNAE / JC	below (unsigned <)	CF=1
JBE / JNA	below or equal	CF=1 or ZF=1

5.3.2 Signed Comparisons (use SF, OF, and ZF)

Mnemonic	Condition	Flags
JG / JNLE	greater (signed >)	ZF=0 and SF=OF
JGE / JNL	greater or equal	SF=OF
JL / JNGE	less (signed <)	SF!=OF
JLE / JNG	less or equal	ZF=1 or SF!=OF

5.3.3 Equality (same for signed and unsigned)

Mnemonic	Condition	Flags
JE / JZ	equal / zero	ZF=1
JNE / JNZ	not equal / not zero	ZF=0

5.3.4 Other

Mnemonic	Condition	Flags
JS	sign (negative)	SF=1
JNS	not sign (non-negative)	SF=0
JO	overflow	OF=1
JNO	no overflow	OF=0

5.4 Why This Matters

After `cmp eax, 0x80`:

- `ja target` treats EAX as **unsigned**. If EAX = 0xFFFFFFFF, that is 4294967295 unsigned, which is above 0x80, so the jump is taken.
- `jg target` treats EAX as **signed**. If EAX = 0xFFFFFFFF, that is -1 signed, which is less than 0x80, so the jump is NOT taken.

The jump mnemonic leaks the programmer's intended signedness of the comparison.

5.5 SETcc and CMOVcc

SETcc sets a byte register to 1 if the condition is true, 0 otherwise:

```
cmp eax, ebx
setg al      ; AL = 1 if EAX > EBX (signed), else 0
```

CMOVcc conditionally moves without branching:

```
cmp eax, ebx
cmovg ecx, edx ; ECX = EDX if EAX > EBX (signed), else ECX unchanged
```

Both use the same condition codes as Jcc.

6. Core Instruction Reference

6.1 Data Movement

6.1.1 MOV – Move

```
mov dst, src      ; dst = src
```

Copies src to dst. Does not affect flags. The most common instruction. Cannot move memory to memory – at least one operand must be a register.

6.1.2 MOVZX – Move with Zero-Extend

```
movzx eax, bl      ; EAX = zero-extend(BL) -- upper 24 bits cleared
movzx eax, WORD PTR [esi] ; EAX = zero-extend(16-bit value at [ESI])
```

Loads a smaller value into a larger register, filling upper bits with zero. Indicates unsigned interpretation.

6.1.3 MOVSX – Move with Sign-Extend

```
movsx eax, bl      ; EAX = sign-extend(BL)
movsx eax, WORD PTR [esi] ; EAX = sign-extend(16-bit value at [ESI])
```

Fills upper bits with the sign bit of the source. Indicates signed interpretation. If BL = 0xFF, then EAX = 0xFFFFFFFF (-1).

6.1.4 XCHG – Exchange

```
xchg eax, ecx ; swap EAX and ECX
```

Atomic swap. Common as a ROP gadget (`xchg eax, esp; ret = stack pivot`). Does not affect flags.

6.1.5 LEA – Load Effective Address

```
lea eax, [ebp-0x40] ; EAX = EBP - 0x40 (compute address, no memory read)
lea eax, [ecx+ecx*2] ; EAX = ECX * 3
```

Computes the address expression and stores the result. Does NOT read memory. Does NOT affect flags. Compilers use it for multi-operand arithmetic.

6.1.6 PUSH – Push onto Stack

```
push eax ; ESP -= 4; [ESP] = EAX
push 0x41414141 ; ESP -= 4; [ESP] = 0x41414141
push DWORD PTR [eax] ; ESP -= 4; [ESP] = [EAX]
```

Decrements ESP by the operand size (4 on x86-32), then writes the operand to [ESP]. Does not affect flags.

6.1.7 POP – Pop from Stack

```
pop eax ; EAX = [ESP]; ESP += 4
```

Reads [ESP] into the destination, then increments ESP by 4. Does not affect flags.

6.1.8 PUSHAD – Push All General Registers

```
pushad ; pushes EAX, ECX, EDX, EBX, original-ESP, EBP, ESI, EDI
```

Pushes all 8 GPRs in the order listed above (EAX first, EDI last). ESP decreases by 32 (8 * 4). The “original ESP” pushed is the value ESP had before PUSHAD began.

After PUSHAD, the stack looks like (top = lowest address):

```
[ESP+0x00] = EDI (pushed last = on top)
[ESP+0x04] = ESI
[ESP+0x08] = EBP
[ESP+0x0C] = original ESP
[ESP+0x10] = EBX
[ESP+0x14] = EDX
[ESP+0x18] = ECX
[ESP+0x1C] = EAX (pushed first = deepest)
```

The CPU pushes in EAX-first order, but because the stack grows downward, the result in memory has EDI at the top. This distinction matters when using PUSHAD in ROP chains.

6.1.9 POPAD – Pop All General Registers

```
popad ; pops EDI, ESI, EBP, (skip ESP), EBX, EDX, ECX, EAX
```

Reverse of PUSHAD. ESP increases by 32. The saved ESP value is discarded (not loaded into ESP).

6.1.10 PUSHFD / POPFD – Push/Pop EFLAGS

```
pushfd ; ESP -= 4; [ESP] = EFLAGS
popfd ; EFLAGS = [ESP]; ESP += 4
```

6.2 Arithmetic

6.2.1 ADD

```
add eax, 4           ; EAX = EAX + 4
add eax, ecx         ; EAX = EAX + ECX
```

Sets OF, SF, ZF, AF, CF, PF.

6.2.2 SUB

```
sub esp, 0x40         ; ESP = ESP - 0x40 (allocate 64 bytes of stack space)
sub eax, ecx         ; EAX = EAX - ECX
```

Sets OF, SF, ZF, AF, CF, PF.

6.2.3 INC / DEC

```
inc eax              ; EAX = EAX + 1
dec ecx              ; ECX = ECX - 1
```

Set OF, SF, ZF, AF, PF. Do NOT modify CF. This matters: `inc` followed by `jc` tests a carry from a *previous* instruction, not from the increment.

6.2.4 NEG – Two’s Complement Negate

```
neg eax              ; EAX = 0 - EAX (= ~EAX + 1)
```

Sets CF=1 unless operand was 0. Useful for converting between positive and negative values without null bytes: `mov eax, 0x1C` encodes nulls (B8 1C 00 00 00), but `mov eax, 0xFFFFFEE4` followed by `neg eax` does not.

6.2.5 MUL – Unsigned Multiply

```
mul ecx              ; EDX:EAX = EAX * ECX (unsigned)
```

One-operand form only. Result is always 64 bits in EDX:EAX. Sets CF and OF if the high half (EDX) is non-zero. ZF, SF, PF are undefined.

6.2.6 IMUL – Signed Multiply

```
imul ecx             ; EDX:EAX = EAX * ECX (signed, one-operand)
imul eax, ecx        ; EAX = EAX * ECX (signed, two-operand)
imul eax, ecx, 12     ; EAX = ECX * 12 (signed, three-operand)
```

Two-operand and three-operand forms store only the low 32 bits. CF and OF set if the result was truncated.

6.2.7 DIV – Unsigned Divide

```
div ecx              ; EAX = EDX:EAX / ECX, EDX = EDX:EAX % ECX
```

Before `div`, EDX must be set up. For 32-bit dividends, `xor edx, edx` (unsigned) or `cdq` (signed, but use `idiv`). Division by zero or quotient overflow raises #DE (divide error exception).

6.2.8 IDIV – Signed Divide

```
cdq                  ; sign-extend EAX into EDX:EAX
idiv ecx            ; EAX = EDX:EAX / ECX (signed), EDX = remainder
```

6.2.9 ADC – Add with Carry

```
adc eax, 0 ; EAX = EAX + 0 + CF
```

Adds source + CF to destination. Used for multi-precision arithmetic.

6.2.10 SBB – Subtract with Borrow

```
sbb eax, eax ; EAX = EAX - EAX - CF = 0 - CF = -CF
```

The `sbb eax, eax` idiom produces 0 (CF=0) or -1 (CF=1). Occasionally seen in branchless code.

6.3 Logic and Bit Manipulation

6.3.1 AND

```
and eax, 0x0F ; EAX = EAX & 0x0F (mask low nibble)
and esp, 0xFFFFF0 ; align ESP to 16-byte boundary
```

Clears CF and OF. Sets SF, ZF, PF.

6.3.2 OR

```
or eax, ecx ; EAX = EAX | ECX
```

Clears CF and OF. `or eax, eax` sets ZF if EAX is zero (same as `test eax, eax` but also writes to EAX).

6.3.3 XOR

```
xor eax, eax ; EAX = 0 (smallest encoding for zeroing a register)
xor eax, ecx ; EAX = EAX ^ ECX
```

Clears CF and OF. `xor eax, eax` is the standard null-free way to zero a register.

6.3.4 NOT – Bitwise Complement

```
not eax ; EAX = ~EAX (flip all bits)
```

Does NOT affect any flags.

6.3.5 TEST

```
test eax, eax ; compute EAX & EAX, set flags, discard result
test al, 0x01 ; check if bit 0 is set
```

Like AND but does not store the result. Clears CF and OF. Sets SF, ZF, PF. Preferred over `cmp eax, 0` for zero checks because it has a shorter encoding.

6.3.6 SHL / SAL – Shift Left

```
shl eax, 1 ; EAX = EAX << 1 (multiply by 2)
shl eax, cl ; EAX = EAX << CL
```

SHL and SAL are identical (both shift left, filling with zeros). CF receives the last bit shifted out. OF set on single-bit shifts if the sign bit changed.

6.3.7 SHR – Shift Right (Logical)

```
shr eax, 4           ; EAX = EAX >> 4 (unsigned divide by 16)
```

Fills vacated high bits with zero. Unsigned shift.

6.3.8 SAR – Shift Right (Arithmetic)

```
sar eax, 1           ; signed divide by 2 (preserves sign bit)
```

Fills vacated high bits with the original sign bit. Signed shift.

6.3.9 ROL / ROR – Rotate Left / Right

```
ror eax, 0x0D         ; rotate EAX right by 13 bits
```

Circular rotation. Common in API hash algorithms (rotate-right-13-add is a widely used hash).

6.3.10 RCL / RCR – Rotate Through Carry

```
rcl eax, 1           ; rotate left through CF (33-bit rotation)
```

Rarely seen outside crypto or multi-precision code.

6.3.11 BSWAP – Byte Swap

```
bswap eax           ; reverse byte order of EAX
```

Converts between big-endian and little-endian. If EAX = 0x41424344, after BSWAP EAX = 0x44434241. Does not affect flags.

6.4 Control Flow

6.4.1 CALL – Call Procedure

```
call 0x00401000       ; push EIP_next; EIP = 0x00401000
call eax              ; push EIP_next; EIP = EAX
call [eax]           ; push EIP_next; EIP = [EAX]
```

Pushes the address of the instruction following CALL onto the stack (ESP -= 4), then transfers control to the target. Does not affect flags.

6.4.2 RET – Return

```
ret                   ; EIP = [ESP]; ESP += 4
ret 0x10              ; EIP = [ESP]; ESP += 4 + 0x10
```

ret pops the return address into EIP. ret N additionally adds N to ESP after popping EIP (callee stack cleanup, used by __stdcall). Does not affect flags.

ret N does NOT pop N bytes before reading EIP. It pops EIP first, then adds N to ESP. The value N cleans up the arguments that the caller pushed.

6.4.3 JMP – Unconditional Jump

```
jmp 0x00401000        ; EIP = 0x00401000
jmp eax              ; EIP = EAX
jmp [eax]            ; EIP = [EAX]
jmp short label       ; relative jump, 8-bit offset
```

6.4.4 Jcc – Conditional Jumps

See Section 5 for the full table. Each Jcc instruction checks EFLAGS and transfers control if the condition is met.

6.4.5 LOOP

```
loop label          ; ECX -= 1; if ECX != 0, jump to label
```

Decrements ECX and jumps if ECX is not yet zero. Does not affect flags. Rarely emitted by optimizing compilers but sometimes useful in shellcode.

6.4.6 JECXZ

```
jecz label         ; if ECX == 0, jump to label (no flag check)
```

6.4.7 INT – Software Interrupt

```
int 3              ; breakpoint trap (0xCC)
int 0x2E           ; system call (Windows, legacy)
```

int 3 is a single-byte instruction (opcode 0xCC) used for breakpoints.

6.4.8 NOP – No Operation

```
nop               ; single-byte NOP (opcode 0x90 = xchg eax, eax)
```

NOP sleds in shellcode use sequences of 0x90. Multi-byte NOPs exist (0F 1F 00, etc.) but the single-byte form is standard for sleds.

6.4.9 LEAVE

```
leave             ; ESP = EBP; POP EBP
```

Equivalent to mov esp, ebp followed by pop ebp. Tears down a stack frame. Common epilogue instruction.

6.5 String and Memory Operations

All string instructions use ESI (source), EDI (destination), ECX (count for rep), and the DF flag (0 = increment, 1 = decrement). Size suffixes: b = byte (1), w = word (2), d = dword (4).

6.5.1 MOVSB / MOVSD – Move String

```
rep movsb        ; copy ECX bytes from [ESI] to [EDI]
rep movsd        ; copy ECX dwords from [ESI] to [EDI]
```

After each element: ESI += size, EDI += size (or -= if DF=1). ECX decremented by rep. Implements memcpy when DF=0.

6.5.2 STOSB / STOSD – Store String

```
rep stosb        ; fill ECX bytes at [EDI] with AL
rep stosd        ; fill ECX dwords at [EDI] with EAX
```

Implements memset. EDI advances; ECX decremented by rep.

6.5.3 LODSB / LODSD – Load String

```
lodsb      ; AL = [ESI]; ESI += 1
lodsd      ; EAX = [ESI]; ESI += 4
```

Loads from [ESI] into the accumulator. Advances ESI. No `rep` prefix is typical – used in loops for byte-by-byte processing (hash computation).

6.5.4 SCASB / SCASD – Scan String

```
repne scasb ; scan [EDI] for AL, stop when found or ECX=0
repe scasd  ; scan [EDI] for mismatch with EAX
```

Compares AL/EAX with [EDI], sets flags, advances EDI. `repne scasb` is the core of `strlen` (scan for null byte).

6.5.5 CMPSB / CMPSD – Compare Strings

```
repe cmpsb ; compare [ESI] with [EDI] byte by byte while equal
```

Compares [ESI] with [EDI], sets flags, advances both. `repe cmpsb` is the core of `memcmp/strcmp`.

6.5.6 REP Prefixes

Prefix	Meaning
<code>rep</code>	Repeat ECX times
<code>repe / repz</code>	Repeat while equal (ZF=1) and ECX > 0
<code>repne / repnz</code>	Repeat while not equal (ZF=0) and ECX > 0

6.5.7 CLD / STD – Clear/Set Direction Flag

```
cld      ; DF = 0 (string ops increment ESI/EDI)
std      ; DF = 1 (string ops decrement ESI/EDI)
```

The Windows ABI requires DF=0 on function entry and exit. Shellcode should execute `cld` before string operations to ensure forward direction. Forgetting this is a common shellcode bug.

6.6 Other Commonly Encountered

6.6.1 CMP

```
cmp eax, 0x80 ; compute EAX - 0x80, set flags, discard result
```

Sets flags as if SUB were performed. The most common comparison instruction. See Section 5 for flag interpretation.

6.6.2 CDQ – Convert Double to Quad

```
cdq      ; EDX = (EAX < 0) ? 0xFFFFFFFF : 0x00000000
```

Sign-extends EAX into EDX:EAX. Required before `idiv`. If EAX is non-negative, EDX becomes 0. Shellcode uses `cdq` after `xor eax, eax` to zero EDX in one byte (opcode 0x99).

6.6.3 CWD – Convert Word to Double

```
cwd      ; DX = sign-extend(AX)
```

6.6.4 CWDE – Convert Word to Double Extended

```
cwde                ; EAX = sign-extend(AX)
```

6.6.5 XLAT – Table Lookup

```
xlat                ; AL = [EBX + AL]
```

Single-byte lookup table instruction. Rarely seen in compiled code.

6.6.6 ENTER

```
enter 0x40, 0        ; push EBP; mov EBP,ESP; sub ESP,0x40
```

Equivalent to the standard prologue but slower. Modern compilers do not emit it.

6.6.7 LEAVE

```
leave               ; ESP = EBP; pop EBP
```

Standard epilogue instruction. Equivalent to `mov esp, ebp; pop ebp`.

7. Stack Fundamentals

7.1 Growth Direction

The x86 stack grows toward **lower** addresses. `push` decrements ESP; `pop` increments ESP. “Top of stack” means the lowest address currently in use.

7.2 Stack Reserve vs. Stack Commit

When a thread is created, Windows **reserves** a large contiguous virtual address range for its stack (default 1 MB) but only **commits** a small portion (typically one or a few pages). As the stack grows, guard pages trigger automatic commit of additional pages. The full reserved range is NOT physically backed at creation time.

Active stack frames are created dynamically as functions are called and destroyed as they return. The total number of functions in the executable does not determine stack size – only the depth of the call chain at any moment.

7.3 State Transitions

7.3.1 PUSH

Before: After push 0x41414141:

ESP = 0x1000	ESP = 0x0FFC
	[0x0FFC] = 0x41414141
[0x1000] = ????	[0x1000] = ????

Rule: ESP = ESP - 4; [ESP] = operand

7.3.2 POP

Before: After pop eax:

ESP = 0x0FFC	ESP = 0x1000
[0x0FFC] = 0x41414141	EAX = 0x41414141

Rule: dst = [ESP]; ESP = ESP + 4

7.3.3 CALL

Before: After call 0x00401000:

EIP = 0x004010A0	EIP = 0x00401000
ESP = 0x1000	ESP = 0x0FFC
	[0x0FFC] = 0x004010A5 (address after CALL)

Rule: ESP -= 4; [ESP] = EIP_next; EIP = target

7.3.4 RET

Before: After ret:

EIP = (address of ret)	EIP = 0x004010A5 (= value popped from stack)
ESP = 0x0FFC	ESP = 0x1000
[0x0FFC] = 0x004010A5	

Rule: EIP = [ESP]; ESP = ESP + 4

7.3.5 RET N (e.g., ret 0x10)

Before: After ret 0x10:

ESP = 0x0FFC	EIP = [0x0FFC] (return address)
[0x0FFC] = retaddr	ESP = 0x0FFC + 4 + 0x10 = 0x1010

Rule: EIP = [ESP]; ESP = ESP + 4 + N

The N bytes cleaned are the callee's arguments (stdcall convention).

7.3.6 SUB ESP, N

Before: After sub esp, 0x40:

ESP = 0x1000	ESP = 0x0FC0
--------------	--------------

Rule: ESP = ESP - N (allocate N bytes of local space)

7.3.7 ADD ESP, N

Before: After add esp, 0x40:

ESP = 0x0FC0	ESP = 0x1000
--------------	--------------

Rule: ESP = ESP + N (deallocate/clean up stack space)

7.4 Guard Pages

The page immediately below the committed portion of the stack is a guard page. Writing to it triggers a STATUS_GUARD_PAGE_VIOLATION exception, which the kernel handles by committing that page and placing a new guard below it. Functions with large local allocations (> 4096 bytes) call `__chkstk` to probe each page in order, ensuring guard pages are triggered sequentially.

8. Function Stack Frames

8.1 Standard Prologue

```
push ebp          ; save caller's frame pointer
mov  ebp, esp     ; establish this function's frame pointer
sub  esp, 0x40    ; allocate 64 bytes for local variables
```

8.2 Standard Epilogue

```
mov  esp, ebp     ; deallocate locals (restore ESP to frame pointer)
pop  ebp          ; restore caller's frame pointer
ret              ; return to caller
```

Or equivalently:

```
leave             ; mov esp, ebp; pop ebp
ret
```

8.3 Frame Layout

Higher addresses (toward stack base)

```
+-----+
| ...           |
| arg2           | [EBP+0x0C]
| arg1           | [EBP+0x08]
| return address | [EBP+0x04] <-- pushed by CALL
| saved EBP      | [EBP+0x00] <-- pushed by prologue
| local var 1    | [EBP-0x04]
| local var 2    | [EBP-0x08]
| ...           |
| local buffer   | [EBP-0x40]
+-----+ <-- ESP (after sub esp, N)
```

Lower addresses (toward stack limit)

Arguments are at positive EBP offsets (+0x08, +0x0C, ...). The gap at +0x04 is the return address. Locals are at negative EBP offsets.

8.4 Frame-Pointer Omission (FPO)

Optimized builds may not use EBP as a frame pointer. Instead:

- EBP is used as a general-purpose register
- All locals and arguments are accessed via ESP-relative offsets
- The frame is harder to read because ESP changes throughout the function (every push/pop/call shifts all offsets)

IDA handles FPO by tracking ESP at each instruction. WinDbg's k command may need symbols or .frame to reconstruct the call stack in FPO code.

8.5 Inlined Functions

Small functions may be inlined by the compiler – their code is inserted directly into the caller, with no call/ret overhead. There is no stack frame for an inlined function.

8.6 Tail Calls

When a function's last action is calling another function and returning the result, the compiler may replace call target; ret with jmp target. The callee reuses the caller's return address. This looks like a jmp to another function

at the end of a function body.

9. Calling Conventions

9.1 cdecl (C Declaration)

- **Arguments:** pushed right-to-left
- **Stack cleanup:** caller (add esp, N after call)
- **Return value:** EAX (or EDX:EAX for 64-bit)
- **Callee-saved:** EBX, ESI, EDI, EBP
- **Caller-saved:** EAX, ECX, EDX

```
; Calling: int result = func(1, 2, 3);
push 3
push 2
push 1
call func
add esp, 0x0C      ; caller cleans 3 args * 4 bytes
; result in EAX
```

Default for C functions. Supports variadic arguments because the caller knows the argument count and cleans up.

9.2 stdcall (Standard Call)

- **Arguments:** pushed right-to-left
- **Stack cleanup:** callee (ret N)
- **Return value:** EAX
- **Callee-saved:** EBX, ESI, EDI, EBP

```
; Calling: VirtualProtect(addr, size, prot, &old);
push offset old_prot ; arg4
push 0x40            ; arg3 = PAGE_EXECUTE_READWRITE
push 0x400           ; arg2 = size
push eax             ; arg1 = address
call VirtualProtect
; no add esp -- callee cleaned up with ret 0x10
; result in EAX
```

Used by all Windows API functions (WINAPI = stdcall). The callee executes `ret 0x10` (4 args * 4 bytes = 0x10).

9.3 fastcall

- **Arguments:** first two in ECX, EDX; remainder pushed right-to-left
- **Stack cleanup:** callee
- **Return value:** EAX

```
; Calling: fastcall_func(1, 2, 3, 4);
push 4 ; arg4
push 3 ; arg3
mov edx, 2 ; arg2 in EDX
mov ecx, 1 ; arg1 in ECX
call fastcall_func
; callee cleans stack args with ret 0x08
```

9.4 thiscall (MSVC C++)

- **Arguments:** this in ECX; rest pushed right-to-left
- **Stack cleanup:** callee
- **Return value:** EAX

```
; Calling: obj->method(arg1, arg2);
push arg2
push arg1
mov ecx, obj_ptr      ; this pointer
call method
```

Looks like stdcall but with ECX loaded before the call. Recognizable by ECX being set to a pointer (not a small integer) immediately before the call.

9.5 On Entry to the Callee

Regardless of convention, when execution reaches the first instruction of the called function:

```
ESP -> return address
ESP+4 -> arg1 (or first stack arg after register args)
ESP+8 -> arg2
ESP+C -> arg3
...
```

After the prologue (push ebp; mov ebp, esp):

```
EBP+0x04 = return address
EBP+0x08 = arg1
EBP+0x0C = arg2
EBP+0x10 = arg3
```

The function does not know or care whether it was reached through a call instruction, a ROP gadget, or an overwritten return address. It consumes state according to the ABI.

10. Normal CALL vs. RET-Based Invocation

10.1 Normal CALL

```
push offset old_prot      ; arg4: lpfl0ldProtect
push 0x40                 ; arg3: flNewProtect
push 0x400                ; arg2: dwSize
push eax                 ; arg1: lpAddress
call VirtualProtect       ; pushes return address, jumps to VirtualProtect
```

After call, the stack seen by VirtualProtect:

```
ESP -> return address (pushed by CALL)
ESP+4 -> lpAddress
ESP+8 -> dwSize
ESP+C -> flNewProtect
ESP+10-> lpfl0ldProtect
```

10.2 RET-Based Invocation (ROP)

Instead of pushing arguments and using call, arrange the stack so that a preceding gadget's ret loads the function address into EIP:

```
ESP -> VirtualProtect      (will become EIP via ret)
+04 ReturnToShellcode     (VirtualProtect's "return address")
+08 ShellcodeAddress      (arg1: lpAddress)
+0C 0x00000400            (arg2: dwSize)
+10 0x00000040            (arg3: flNewProtect)
+14 WritableAddress        (arg4: lpfl0ldProtect)
```

When the preceding gadget executes ret:

1. EIP = [ESP] = VirtualProtect address
2. ESP += 4 (now points at ReturnToShellcode)

VirtualProtect now sees exactly the same stack layout it would see after a normal `call`:

```
ESP    -> ReturnToShellcode  (its "return address")
ESP+4  -> ShellcodeAddress   (arg1)
ESP+8  -> 0x400              (arg2)
ESP+C  -> 0x40               (arg3)
ESP+10-> WritableAddress     (arg4)
```

VirtualProtect executes, changes page protections, then does `ret 0x10` (stdcall cleanup):

1. EIP = [ESP] = ReturnToShellcode
2. ESP += 4 + 0x10 = skips past the 4 arguments

Execution lands at the shellcode with the page now marked executable.

The CPU does not distinguish how VirtualProtect was reached. It follows the ABI mechanically: arguments are at fixed offsets from ESP, and `ret N` cleans them.

11. Buffers and Stack Overflows

11.1 Stack Buffer Layout

Higher addresses

```
+-----+
| function arguments      | [EBP+0x0C], [EBP+0x08]
+-----+
| return address         | [EBP+0x04]
+-----+
| saved EBP              | [EBP+0x00]
+-----+
| /GS cookie (if present) | [EBP-0x04]
+-----+
| other local variables   |
+-----+
| local buffer (e.g. 64 B) | [EBP-0x40]
+-----+ <-- ESP
```

Lower addresses

11.2 Overflow Mechanics

An unbounded copy (`strcpy`, `sprintf`, `recv` without length check) writes past the buffer boundary, overwriting adjacent locals, saved EBP, and the return address. The write travels from low addresses toward high addresses (upward in the diagram).

11.3 Offset Calculation

The offset from the start of the buffer to the return address:

```
offset = (EBP - &buffer) + 4
```

If buffer is at `[EBP-0x40]`:

```
offset = 0x40 + 4 = 0x44 = 68 bytes
```

The + 4 accounts for the saved EBP between the locals and the return address.

Verify statically by reading the `lea reg, [ebp-X]` that feeds the copy. The offset comes from the buffer's actual position relative to EBP, not from the frame size (`sub esp, N`).

11.4 Cyclic Pattern Verification

1. Generate a cyclic (De Bruijn) pattern of sufficient length
2. Send the pattern through the vulnerability
3. Read the value in EIP at the crash
4. Look up the 4-byte sequence in the pattern to get the exact offset

```
# Using mona in WinDbg:
# !py mona pattern_create 500
# !py mona pattern_offset <EIP_value>
```

If the pattern offset matches your static calculation, the offset is confirmed. If they differ, the difference reveals saved registers, cookies, or alignment padding you missed.

11.5 Bad Characters

Certain byte values are transformed or truncated by the target application before they reach the vulnerable buffer. Common bad characters:

- 0x00 – null terminator (truncates strings)
- 0x0A – line feed
- 0x0D – carriage return
- 0x09 – tab
- 0x20 – space

Test by sending all 256 byte values through the vulnerability and comparing what arrives in memory with what was sent. Any byte that is missing, modified, or causes truncation is a bad character.

Bad characters affect: - The return address bytes (the address you overwrite EIP with) - ROP gadget addresses - Shellcode bytes - All payload data that passes through the vulnerable path

11.6 Partial Overwrites

Sometimes only the low 1, 2, or 3 bytes of the return address can be controlled (the overflow is bounded, or null bytes terminate the write early). A partial overwrite can redirect EIP within the same module if the high bytes of the original return address are preserved.

11.7 Stack Pivots

When the shellcode or ROP chain is not adjacent to the overwritten return address (e.g., it lives in a different buffer), a stack pivot redirects ESP to point at the controlled data:

```
xchg eax, esp ; ret      ; ESP = old EAX (must point to ROP chain)
mov esp, eax ; ret      ; same effect
add esp, 0x800 ; ret     ; jump ESP forward to reach data further up the stack
```

11.8 SEH Overwrites

When /GS cookies block a direct return-address overwrite, structured exception handling (SEH) records on the stack can be targeted instead. The SEH chain is a linked list starting at FS:[0]. Overwriting the handler pointer in an EXCEPTION_REGISTRATION_RECORD and then triggering an exception diverts execution through the corrupted handler. See the SEH documentation in this repository for the full technique.

12 12. Arithmetic Used in Exploit Development

12.1 Offset Arithmetic

Stack offsets, buffer sizes, and pointer adjustments are constant arithmetic:

```
add esp, 0x20      ; advance ESP past 32 bytes of padding
sub esp, 0x40      ; allocate 64 bytes of local space
```

12.2 Two's Complement

Negative numbers are represented as two's complement. The value -1 is 0xFFFFFFFF. To negate: invert all bits and add 1.

```
0x0000001C = 28
-0x0000001C = ~0x0000001C + 1 = 0xFFFFFFFF + 1 = 0xFFFFFFE4
```

```
neg eax ; EAX = 0 - EAX
```

12.3 Null-Byte Avoidance via Negative Arithmetic

When an immediate is too large for a sign-extended imm8 (outside -128..+127), the assembler uses a 32-bit immediate that may contain null bytes. `add esp, 0x80` encodes as `81 C4 80 00 00 00` (null bytes). Use the equivalent subtraction of the negated value:

```
sub esp, -0x80 ; ESP = ESP - (-0x80) = ESP + 0x80
; -0x80 fits in a signed byte, encodes as 83 EC 80
```

Note: small values like `add esp, 0x20` already use the 3-byte imm8 encoding (`83 C4 20`) with no null bytes, so this technique is only needed for values outside the signed-byte range.

Or split the value:

```
; Need EAX = 0x00000040 (contains nulls)
mov eax, 0x80808080
add eax, 0x7F7F7FC0 ; 0x80808080 + 0x7F7F7FC0 = 0x00000040
```

12.4 Alignment

```
and esp, 0xFFFFFFFF ; round ESP down to nearest 16-byte boundary
```

This clears the low 4 bits. Used for SSE alignment requirements and before calling functions that expect aligned stacks.

12.5 Integer Truncation and Extension

```
movzx eax, al ; zero-extend 8-bit to 32-bit (unsigned)
movsx eax, al ; sign-extend 8-bit to 32-bit (signed)
```

A `mov al, value` followed by operations on EAX can produce unexpected results if the upper bytes of EAX were not cleared. Always check whether the code uses `movzx/movsx` to properly extend the value.

12.6 Wraparound

32-bit arithmetic wraps at 0xFFFFFFFF:

```
0xFFFFFFFF + 1 = 0x00000000 (CF=1)
0x00000000 - 1 = 0xFFFFFFFF (CF=1)
```

This affects size calculations and can be exploited (integer overflow leading to undersized allocation leading to heap/stack overflow).

13. ROP Gadget Semantics

A gadget is a short instruction sequence ending in `ret` (or equivalent). The `ret` at the end pops the next address from the stack, chaining execution to the next gadget. Each gadget performs one small state change.

13.1 Register Load

```
pop eax ; ret
```

Before:	After:
ESP -> 0x11111111	EAX = 0x11111111
NextGadget	EIP = NextGadget
	ESP = old_ESP + 8

Loads an immediate value from the stack into a register. Consumes 8 bytes of stack (4 for the value, 4 for the return address used by ret).

13.2 Register Exchange

```
xchg eax, ecx ; ret
```

Before:	After:
EAX = A, ECX = C	EAX = C, ECX = A
ESP -> NextGadget	EIP = NextGadget
	ESP = old_ESP + 4

13.3 Dereference (Memory Read)

```
mov eax, [eax] ; ret
```

Before:	After:
EAX = 0x76300000	EAX = [0x76300000] (value at that address)
ESP -> NextGadget	EIP = NextGadget

Used to read IAT entries, data pointers, or structure fields. Precondition: EAX must hold a valid readable address.

13.4 Memory Write

```
mov [edi], eax ; ret
```

Before:	After:
EDI = target_addr	[target_addr] = EAX
EAX = value_to_write	EIP = NextGadget
ESP -> NextGadget	

The fundamental store primitive for building a fake stack frame. Precondition: EDI must hold a valid writable address.

13.5 Arithmetic

```
add eax, ecx ; ret
```

Before:	After:
EAX = X, ECX = Y	EAX = X + Y
	EIP = NextGadget

Used to compute values that cannot be loaded directly (e.g., because they contain null bytes).

13.6 Stack Adjustment

```
add esp, 0x20 ; ret
```

Before:	After:
ESP -> 0x1000	ESP = 0x1000 + 0x20 + 4 = 0x1024

(skip 0x20 bytes, then ret pops next 4)

Skips over unwanted stack data. Common when gadgets have trailing pops or when aligning to a specific stack location.

13.7 Stack Pivot

```
xchg eax, esp ; ret
```

Before:	After:
EAX = 0x0C0C0C0C	ESP = 0x0C0C0C0C
	EIP = [0x0C0C0C0C] (first gadget at pivoted stack)

Redirects the entire ROP chain to a different memory region. Precondition: EAX must point to controlled data structured as a ROP chain.

```
mov esp, ebp ; pop ebp ; ret
```

Uses EBP as the pivot source. The pop ebp consumes one DWORD, so aim EBP 4 bytes before the API address so the stray pop eats a dummy value and ESP lands on the target.

13.8 PUSHAD in ROP

```
pushad ; ret
```

Pushes all 8 registers (32 bytes) then ret pops the next address. The pre-PUSHAD register arrangement determines the fake call frame on the stack. Because PUSHAD pushes EAX first (deepest) and EDI last (on top), and ret pops from the top, EDI's value becomes the next EIP.

To use PUSHAD for a VirtualProtect call, arrange registers so the resulting stack frame contains the correct arguments at the correct offsets.

13.9 Conditional Gadgets

Some gadgets contain conditional logic:

```
test eax, eax
jne skip
pop ecx
skip:
ret
```

These are fragile and version-specific. Prefer unconditional gadgets where possible.

13.10 Side Effects

Every gadget may clobber registers or flags beyond its intended operation. Document the full before/after state, including which registers are destroyed. A gadget like mov [edi], eax ; pop esi ; ret clobbers ESI – if ESI was holding a needed value, it must be reloaded.

14 14. PUSHAD-Based API Calls

14.1 PUSHAD Push Order and Memory Layout

PUSHAD pushes registers in this CPU-defined order:

1. EAX (pushed first, ends up deepest in memory)
2. ECX
3. EDX
4. EBX
5. original ESP (value before PUSHAD)

6. EBP
7. ESI
8. EDI (pushed last, ends up on top of stack)

Because the stack grows downward, the resulting memory layout is:

```
ESP+0x00 -> EDI      (top of stack -- lowest address)
ESP+0x04 -> ESI
ESP+0x08 -> EBP
ESP+0x0C -> orig ESP
ESP+0x10 -> EBX
ESP+0x14 -> EDX
ESP+0x18 -> ECX
ESP+0x1C -> EAX      (deepest -- highest address)
```

14.2 Using PUSHAD to Build a Call Frame

For a pushad; ret gadget to invoke VirtualProtect via ROP, arrange registers before PUSHAD so the stack afterward looks like a valid stdcall frame:

Target stack layout after pushad:

```
ESP+0x00 -> EDI = VirtualProtect address  <-- ret pops this into EIP
ESP+0x04 -> ESI = return address           <-- VP's return address
ESP+0x08 -> EBP = arg1 (lpAddress)
ESP+0x0C -> orig ESP = arg2 (dwSize)       <-- cannot control directly
ESP+0x10 -> EBX = arg3 (flNewProtect)
ESP+0x14 -> EDX = arg4 (lpflOldProtect)
```

The ret after PUSHAD pops EDI into EIP, so EDI must hold the function address. ESI becomes the return address. EBP becomes arg1. EBX and EDX become args 3 and 4. The original ESP value occupies the arg2 slot and may need to be set to a useful value (often a large-enough size works because ESP typically points within the stack region being marked executable).

15 15. DEP and Page Protections

15.1 Executable vs. Non-Executable Memory

DEP (Data Execution Prevention) / NX (No-eXecute) marks memory pages as non-executable. Code on the stack or heap will fault when the CPU attempts to fetch instructions from those pages.

15.2 Why Stack Shellcode Faults

Without DEP: overflow the return address to point at shellcode on the stack; the CPU executes it.

With DEP: the stack page is marked NX. When EIP points into the stack, the CPU raises an access violation on the next instruction fetch, not on data access.

15.3 VirtualProtect

```
BOOL VirtualProtect(
    LPVOID lpAddress,           // address in the region to change
    SIZE_T dwSize,             // size of the region
    DWORD  flNewProtect,        // new protection constant
    PDWORD lpflOldProtect      // pointer to DWORD receiving old protection
);
```

Changes the protection on committed pages in the calling process. Used in DEP bypass to mark the shellcode's page as executable.

lpflOldProtect must point to writable memory. VirtualProtect writes the old protection value to this address. If the pointer is invalid, the call fails. A common writable location is any address in a writable data section (e.g., `.data` or a stack address).

VirtualProtect operates on page granularity internally (4 KB pages), but the caller does not need to provide a page-aligned `lpAddress`. The API rounds down to the page boundary containing `lpAddress` and rounds up to cover all pages spanned by the range `[lpAddress, lpAddress + dwSize)`.

15.4 VirtualAlloc

```
LPVOID VirtualAlloc(
    LPVOID lpAddress,      // desired address (or NULL for auto)
    SIZE_T dwSize,         // size of the region
    DWORD  flAllocationType, // allocation type
    DWORD  flProtect       // protection
);
```

Allocates new memory with the specified protection. Can be used to allocate executable memory and copy shellcode into it.

15.5 Protection Constants

Constant	Value	Meaning
PAGE_EXECUTE_READWRITE	0x40	Read, write, and execute
PAGE_EXECUTE_READ	0x20	Read and execute
PAGE_READWRITE	0x04	Read and write (no execute)
PAGE_READONLY	0x02	Read only
PAGE_NOACCESS	0x01	No access

15.6 Allocation Type Constants

Constant	Value	Meaning
MEM_COMMIT	0x1000	Commit pages (back with physical storage)
MEM_RESERVE	0x2000	Reserve address space without committing
MEM_COMMIT MEM_RESERVE	0x3000	Reserve and commit in one call

15.7 WinDbg Verification

```
0:000> !vprot <address>
BaseAddress: 00c0f000
AllocationBase: 00b70000
RegionSize: 00001000
State: 00001000 MEM_COMMIT
Protect: 00000040 PAGE_EXECUTE_READWRITE
Type: 00020000 MEM_PRIVATE
```

Before the bypass: `Protect = 0x04 (PAGE_READWRITE)`. After `VirtualProtect`: `Protect = 0x40 (PAGE_EXECUTE_READWRITE)`.

16. ASLR and Module Addressing

16.1 Image Base and RVA

Every PE module has a **preferred image base** (e.g., `0x00400000` for executables, `0x10000000` for DLLs). If loaded at that address, all pointers in the binary work without modification.

A **Relative Virtual Address (RVA)** is an offset from the module's loaded base:

$VA = \text{ModuleBase} + \text{RVA}$
 $\text{RVA} = VA - \text{ModuleBase}$

If the module loads at a different base (relocation), the loader patches absolute addresses using the relocation table.

16.2 ASLR

Address Space Layout Randomization randomizes the base address of modules, stack, and heap. The stack and heap are randomized per process launch. System DLL bases (kernel32.dll, ntdll.dll, etc.) are randomized once at boot time and remain the same across all processes until the next reboot. Application EXEs with /DYNAMICBASE may be randomized per launch. With ASLR:

- Gadget addresses change (per-boot for system DLLs, per-launch for EXEs)
- Hardcoded addresses in exploits break
- A leaked DLL base is valid for all processes on the same machine until reboot

16.3 Fixed vs. Randomized Modules

Not all modules opt in to ASLR. Modules compiled without /DYNAMICBASE in their PE header load at their preferred base every time. These are the modules from which ROP gadgets can reliably be taken.

Check in WinDbg:

```

0:000> !nmmod                      (with nml extension)
0:000> lm m modulename             (check base address)

```

16.4 Exploiting Non-ASLR Modules

1. Identify a module loaded at a fixed base (application-shipped DLLs are common candidates)
2. Verify its base address is consistent across runs
3. Extract gadgets using the fixed RVAs
4. Compute gadget VAs: $\text{gadget_VA} = \text{fixed_base} + \text{gadget_RVA}$

16.5 Partial Pointer Overwrites

If only the low 2 bytes of a return address can be overwritten, ASLR does not protect the full address – the low 16 bits of a module's base are often predictable (e.g., page-aligned). This narrows the entropy enough for a probabilistic attack.

16.6 Information Leaks

If the application leaks a pointer (via format string, error message, or protocol response), the attacker can compute the module base at runtime:

```

leaked_VA = base + known_RVA
base = leaked_VA - known_RVA

```

With the base known, all RVAs resolve to correct VAs despite ASLR.

17 17. Windows Process Structures and PEB Walking

17.1 The Path

```

FS:[0x30] -> PEB
PEB+0x0C -> PEB_LDR_DATA
PEB_LDR_DATA+0x14 -> InMemoryOrderModuleList (Flink)

```

17.2 TEB (Thread Environment Block)

The FS segment register base points to the TEB. Key offsets:

Offset	Field
FS:[0x00]	EXCEPTION_REGISTRATION_RECORD pointer (SEH chain head)
FS:[0x04]	StackBase (top of stack, highest address)
FS:[0x08]	StackLimit (bottom of committed stack)
FS:[0x18]	Self (linear address of TEB)
FS:[0x30]	PEB pointer

17.3 PEB (Process Environment Block)

Offset	Field
+0x02	BeingDebugged (BOOLEAN)
+0x08	ImageBaseAddress
+0x0C	Ldr (PEB_LDR_DATA pointer)

17.4 PEB_LDR_DATA

Offset	Field
+0x0C	InLoadOrderModuleList.Flink
+0x14	InMemoryOrderModuleList.Flink
+0x1C	InInitializationOrderModuleList.Flink

17.5 _LDR_DATA_TABLE_ENTRY

This structure contains information about each loaded module. The three lists thread through this structure at different offsets:

Field	Offset from entry base
InLoadOrderLinks	+0x00
InMemoryOrderLinks	+0x08
InInitializationOrderLinks	+0x10
DllBase	+0x18
EntryPoint	+0x1C
SizeOfImage	+0x20
FullDllName	+0x24
BaseDllName	+0x2C

17.6 List Entry Pointers and Structure Offsets

The Flink pointer in a linked list entry points to the **LIST_ENTRY** field in the next structure, NOT to the base of the next structure. To reach the base of the containing structure, subtract the field's offset:

List	Links field offset	Subtract to reach entry base
InLoadOrder (+0x0C)	+0x00	0x00
InMemoryOrder (+0x14)	+0x08	0x08
InInitializationOrder (+0x1C)	+0x10	0x10

For InMemoryOrderModuleList: the Flink points to `next_entry + 0x08`. To access DllBase (at +0x18 from entry base), you can read `[Flink + 0x10]` (because $0x18 - 0x08 = 0x10$).

17.7 Assembly Walkthrough

```

; Get PEB
xor ecx, ecx
mov eax, fs:[ecx+0x30]      ; EAX = PEB

; Get PEB_LDR_DATA
mov eax, [eax+0x0C]        ; EAX = PEB->Ldr

; Get first entry in InMemoryOrderModuleList
mov esi, [eax+0x14]        ; ESI = Flink (points to first entry's InMemoryOrderLinks)

; Walk the list to find kernel32.dll
; ESI points to InMemoryOrderLinks (offset +0x08 within the entry)
; DllBase is at entry+0x18, which is [ESI+0x10] from the links pointer

next_module:
mov ebx, [esi+0x10]        ; EBX = DllBase
mov edi, [esi+0x28]        ; EDI = BaseDllName.Buffer (Unicode string)
                           ; BaseDllName is at entry+0x2C; from ESI (at entry+0x08)
                           ; that is 0x24, plus 0x04 to skip the UNICODE_STRING header
mov esi, [esi]             ; ESI = Flink (next entry)
                           ; compare name at EDI with target module name...
                           ; if match: EBX = module base

```

17.8 Module Order (Windows 7/10 32-bit, typical)

Using InMemoryOrderModuleList: 1. The executable itself 2. ntdll.dll 3. kernel32.dll (or KernelBase.dll on newer systems)

Using InInitializationOrderModuleList: 1. ntdll.dll 2. kernel32.dll 3. KernelBase.dll

The exact order can vary by Windows version. Shellcode that assumes “the second entry is always kernel32” is fragile. Comparing module names is more reliable.

17.9 WinDbg Verification

```

0:000> !peb
0:000> !teb
0:000> dt ntdll!_PEB @$peb
0:000> dt ntdll!_PEB_LDR_DATA poi(@$peb+0xc)
0:000> !dlls

```

18. PE Structures and Export Resolution

18.1 Header Chain

```

DllBase
+0x00  IMAGE_DOS_HEADER
+0x00  e_magic      ("MZ" = 0x5A4D)
+0x3C  e_lfanew     (offset to NT headers)

DllBase + e_lfanew
+0x00  Signature    ("PE\0\0" = 0x00004550)
+0x04  IMAGE_FILE_HEADER (20 bytes)
+0x18  IMAGE_OPTIONAL_HEADER32
+0x60  DataDirectory[0] = Export Directory
+0x00  VirtualAddress (RVA of IMAGE_EXPORT_DIRECTORY)

```

+0x04 Size

Export Directory RVA is at: DllBase + e_lfanew + 0x78

18.2 IMAGE_EXPORT_DIRECTORY

Offset	Field	Meaning
+0x18	NumberOfNames	Count of named exports
+0x1C	AddressOfFunctions	RVA of function address table (array of RVAs)
+0x20	AddressOfNames	RVA of name pointer table (array of RVAs to strings)
+0x24	AddressOfNameOrdinals	RVA of ordinal table (array of 16-bit ordinals)

18.3 Resolution Algorithm

1. **Locate module base** (from PEB walk or known address)
2. **Read e_lfanew:** pe_offset = [base + 0x3C]
3. **Locate PE header:** pe_header = base + pe_offset
4. **Locate export directory RVA:** export_rva = [pe_header + 0x78]
5. **Compute export directory VA:** exports = base + export_rva
6. **Get table pointers:**
 - names = base + [exports + 0x20]
 - ordinals = base + [exports + 0x24]
 - functions = base + [exports + 0x1C]
7. **Iterate names:** For index i, name string is at base + [names + i*4]
8. **Match target name** (compare string or hash)
9. **Get ordinal:** ordinal = [ordinals + i*2] (16-bit value)
10. **Get function RVA:** func_rva = [functions + ordinal*4]
11. **Compute function VA:** func_va = base + func_rva

18.4 Assembly Implementation

```
; Assume EBX = module base
mov  edx, [ebx+0x3C]      ; e_lfanew
add  edx, ebx             ; PE header VA
mov  edx, [edx+0x78]      ; export directory RVA
add  edx, ebx             ; export directory VA
mov  ecx, [edx+0x18]      ; NumberOfNames
mov  eax, [edx+0x20]      ; AddressOfNames RVA
add  eax, ebx             ; AddressOfNames VA

; Loop through names
find_function:
    dec  ecx
    mov  esi, [eax+ecx*4]  ; name RVA
    add  esi, ebx          ; name VA
    ; compare string at ESI with target (or compute hash)
    ; if match: ecx = index
    ; ...

; Resolve address using ordinal
mov  eax, [edx+0x24]      ; AddressOfNameOrdinals RVA
add  eax, ebx
movzx ecx, word ptr [eax+ecx*2] ; ordinal
```

```

mov eax, [edx+0x1C]    ; AddressOfFunctions RVA
add eax, ebx
mov eax, [eax+ecx*4]    ; function RVA
add eax, ebx           ; function VA

```

18.5 Forwarded Exports

If a function RVA falls within the export directory's address range, it is a **forwarded export** – the RVA points to a string like "NTDLL.RtlAllocateHeap" instead of code. The resolver must parse this string, load the target module, and re-resolve. Shellcode that encounters a forward must handle it or avoid APIs known to be forwarded.

18.6 API Sets (Conceptual)

On Windows 8+, some DLL names are virtualized through API sets (e.g., `api-ms-win-core-*.dll` maps to `KernelBase.dll`). The loader resolves these transparently, but shellcode that walks the module list may encounter API set entries. They are not real DLLs – the actual code lives in a backing DLL (typically `KernelBase.dll` or `ntdll.dll`).

19 19. Position-Independent Shellcode

19.1 Core Constraints

Position-independent shellcode must work at any load address:

- **No absolute code addresses** – cannot reference labels with fixed addresses
- **No absolute data addresses** – cannot use global string pointers
- **No import table** – must resolve APIs at runtime (PEB walking)

19.2 Self-Location: JMP-CALL-POP

The standard technique to discover the shellcode's own address:

```

jmp short get_addr    ; forward jump over the call
execute:
  pop esi              ; ESI = address of "data" below
  ; ... shellcode body using ESI as base ...

get_addr:
  call execute          ; pushes address of next instruction onto stack
  ; "data" starts here (strings, encoded payloads, etc.)
  db "cmd.exe", 0

```

`call execute` pushes the address of the byte after the `call` instruction (the start of "data") onto the stack. `pop esi` captures that address.

19.3 Stack Strings

Build strings on the stack to avoid embedding them at a fixed data address:

```

xor eax, eax          ; EAX = 0
push eax              ; null terminator
push 0x636C6163        ; "calc" (little-endian, reversed)
mov ebx, esp          ; EBX = pointer to "calc\0" on the stack

```

Byte-by-byte: `push 0x636C6163` places bytes `63 61 6C 63` in memory (little-endian), which reads as the ASCII string `calc`.

19.4 Null-Byte Avoidance

Null bytes (`0x00`) terminate C strings and are bad characters in most buffer overflows. Techniques:

```

; Instead of: mov eax, 0          (encodes 0x00000000)
xor  eax, eax                    ; same result, no null bytes

; Instead of: push 0             (encodes 0x00000000 as immediate)
xor  eax, eax
push eax                        ; push the zeroed register

; Instead of: mov eax, 0x00401000 (contains 0x00)
; Use arithmetic or encoding to avoid the null byte

```

19.5 Direction Flag

The Windows ABI expects DF=0 (forward direction for string ops). If DF might be set from prior execution, clear it at the start of the shellcode:

```
cld          ; DF = 0
```

Forgetting `cld` can cause `rep movsb` to copy backward, corrupting the destination or segfaulting.

19.6 Register Preservation

Some shellcode contexts (SEH handlers, egghunters) require specific registers to be preserved or set to particular values on entry. Document assumptions about register state at the start of your shellcode.

19.7 Stack Safety

Shellcode runs on the thread's existing stack. If the shellcode's stack usage overlaps with the ROP chain or the exploit payload sitting on the stack, it will corrupt its own data. Solutions:

- `sub esp, N` at the start to create clearance below
- Use registers instead of the stack where possible
- Be aware of how deep API calls push before returning

20. API Hashing

20.1 Why Hash?

Comparing full API name strings requires embedding those strings in the shellcode (space-expensive) or finding them at runtime. Hashing reduces each API name to a 32-bit constant:

1. Compute a hash of each exported name during the PEB/PE walk
2. Compare the hash against a known target constant
3. When matched, use the corresponding function address

20.2 Common Hash Algorithm: ROR-13-ADD

```

compute_hash:
  xor  edx, edx          ; hash = 0
  xor  ecx, ecx
hash_loop:
  mov  cl, [esi]          ; load next byte of function name
  test cl, cl
  jz   hash_done          ; stop at null terminator
  ror  edx, 0x0D          ; rotate right by 13
  add  edx, ecx           ; add character
  inc  esi
  jmp  hash_loop

```

```
hash_done:
    ; EDX = computed hash
```

The rotation constant (13) is arbitrary but widely used. Different shellcode projects use different constants.

20.3 Case Sensitivity

Export names are case-sensitive. If the hash algorithm does not normalize case, the target constant must match the exact casing in the export table. Some implementations convert to uppercase before hashing to handle inconsistencies.

20.4 Collision Handling

Hash collisions are possible – two different function names producing the same 32-bit hash. In practice, collisions within a single module's export table are rare with a good rotation constant. If collisions are a concern, use a wider hash or add the module name to the hash computation.

20.5 Module + Function Hashing

Some implementations hash the module name (e.g., "kernel32.dll") and the function name separately, then combine them (add, xor, or concatenate) to produce a single lookup key. This avoids matching the wrong function in the wrong module.

20.6 Finding Hash Constants

To use a hash-based resolver, you need the precomputed hash for each target API. Compute it offline:

```
def ror13_hash(name: bytes) -> int:
    h = 0
    for b in name:
        h = ((h >> 13) | (h << (32 - 13))) & 0xFFFFFFFF
        h = (h + b) & 0xFFFFFFFF
    return h

print(hex(ror13_hash(b"WinExec")))      # example
print(hex(ror13_hash(b"ExitProcess")))  # example
```

21. String Construction in Shellcode

21.1 Pushing Strings Backward

Strings are built on the stack by pushing DWORDs in reverse order (last 4 characters first):

```
; Build "calc.exe\0" on the stack
xor eax, eax
push eax                ; null terminator (4 zero bytes)
push 0x6578652E         ; ".exe" -> 2E 65 78 65
push 0x636C6163         ; "calc" -> 63 61 6C 63
mov ebx, esp            ; EBX -> "calc.exe\0"
```

Memory at ESP after these pushes:

```
ESP+0x00: 63 61 6C 63  "calc"
ESP+0x04: 2E 65 78 65  ".exe"
ESP+0x08: 00 00 00 00  "\0\0\0\0"
```

Read left to right, this is calc.exe\0 followed by three padding zeros.

21.2 How to Compute the DWORD Values

Take the target string, split into 4-byte chunks, reverse each chunk's byte order for little-endian:

```
String: "calc"
ASCII:  63 61 6C 63
DWORD (little-endian): 0x636C6163
```

```
String: ".exe"
ASCII:  2E 65 78 65
DWORD (little-endian): 0x6578652E
```

21.3 Handling Odd Lengths

If the string length is not a multiple of 4, pad the last chunk. If the padding bytes would be zero and zeros are bad characters, use a register write instead:

```
; "cmd\0" = 63 6D 64 00 -> problem: contains 0x00
; Solutions:
; 1. Push as part of the null terminator push (if aligned)
; 2. Use XOR encoding
; 3. Write the last byte separately:
push 0x00646D63          ; only works if 0x00 is not a bad char
; Or:
push 0x01646D63          ; push with dummy byte
mov byte ptr [esp+3], 0   ; zero out the dummy (but 0x00 in the instruction!)
; Or use xor:
xor eax, eax
mov [esp+3], al           ; AL = 0 from xor, instruction encodes without literal 0x00
```

21.4 Avoiding Bad Characters in Strings

If any byte of the string is a bad character, XOR-encode the string and decode at runtime:

```
push 0x6578652E ^ 0x41414141 ; XOR-encoded ".exe"
xor dword ptr [esp], 0x41414141 ; decode in place
```

21.5 Unicode (UTF-16LE) Strings

Windows API functions with W suffix expect UTF-16LE strings. Each ASCII character becomes 2 bytes (character + 0x00):

```
"cmd" in UTF-16LE: 63 00 6D 00 64 00 00 00
```

This is impractical to push directly because of embedded null bytes. Use the A (ANSI) versions of APIs when possible, or build the Unicode string byte-by-byte.

22. Format-String Assembly Concepts

22.1 Variadic Arguments on x86

printf-family functions use cdecl with variadic arguments:

```
push arg3          ; 3rd format argument
push arg2          ; 2nd format argument
push arg1          ; 1st format argument
push offset fmt_string ; format string
call printf
add esp, 0x10       ; caller cleans up
```

The format string parser walks up the stack, consuming arguments based on format specifiers. Each `%x`, `%s`, `%n`, etc. reads the next DWORD from the stack.

22.2 Format Specifiers

Specifier	Action
<code>%x</code>	Read DWORD from stack, print as hex
<code>%s</code>	Read DWORD from stack as pointer, print string at that address
<code>%n</code>	Read DWORD from stack as pointer, write character count to that address
<code>%hn</code>	Same as <code>%n</code> but writes only 16 bits (short)
<code>%hhn</code>	Same as <code>%n</code> but writes only 8 bits (char)

22.3 Why Format Strings Are Dangerous

If the format string itself is attacker-controlled:

```
printf(user_input);    // vulnerable: user_input IS the format string
```

The attacker controls which specifiers are processed. `%x` reads stack values (information disclosure). `%n` writes to memory (arbitrary write).

22.4 Parameter Indexing

Direct parameter access allows targeting a specific stack argument:

```
%5$x    -- print the 5th argument as hex
%5$n    -- write character count to address pointed to by 5th argument
```

This avoids needing to pop through intervening arguments with dummy `%x` specifiers.

22.5 Write-What-Where with `%n`

`%n` writes the **number of characters printed so far** to the address pointed to by the corresponding argument. By controlling the character count and the target address:

1. Place the target address on the stack (often in the format string itself)
2. Use `%Nc` (print N characters of padding) to control the character count
3. Use `%n` (or `%hn` / `%hhn`) to write the count to the target address

22.6 Partial Writes with `%hn`

Writing a full 32-bit value with `%n` requires printing billions of characters. Instead, split the target value into two 16-bit halves and write each with `%hn`:

```
Target value: 0xAABBCCDD
Write 0xCCDD to target_addr    using %hn (after printing 0xCCDD chars)
Write 0xAABB to target_addr+2  using %hn (after printing 0xAABB chars total)
```

The character count is cumulative across the entire `printf` call. To write the second half, compute the additional padding needed:

```
second_padding = target_high - current_count (mod 0x10000 if wraparound needed)
```

22.7 `%hhn` for Byte-Granularity Writes

`%hhn` writes a single byte (the low 8 bits of the character count). Four `%hhn` writes can construct any 32-bit value:

```

Write byte 0 to addr+0    (count mod 256 = target_byte_0)
Write byte 1 to addr+1    (count mod 256 = target_byte_1)
Write byte 2 to addr+2    (count mod 256 = target_byte_2)
Write byte 3 to addr+3    (count mod 256 = target_byte_3)

```

22.8 Little-Endian Address Placement

When the format string itself contains target addresses (for %n to use as pointers), those addresses are placed in the format string buffer in little-endian byte order. The format string parser reads them as DWORDs from the stack.

23 23. IDA Pro Reading Guide

23.1 Views

- **Disassembly (text):** linear listing of instructions with addresses, opcodes, and operands
- **Graph view:** control-flow graph with basic blocks as nodes and branches as edges; the primary view for understanding function logic
- **Pseudocode (F5):** decompiler output approximating C; useful as a hypothesis but can be wrong – always verify against the actual assembly

23.2 Stack Variables

IDA labels stack variables in the function frame. A variable shown as `var_40` typically means `[EBP-0x40]` (or `[ESP+offset]` in FPO code). IDA's variable naming is a convenience label, not ground truth – verify the actual offset by reading the instruction operand.

23.3 Argument Labels

Function parameters are labeled `arg_0` (`[EBP+0x08]`), `arg_4` (`[EBP+0x0C]`), etc. The offset names reflect the displacement from EBP, not the argument's position in the prototype.

23.4 Cross-References (Xrefs)

Press X on a function or variable to see all locations that reference it. Xrefs reveal:

- Who calls this function
- Where a global variable is read or written
- What code references a string

Follow xrefs backward to trace data flow from a sink to a source.

23.5 Imported Functions

The Imports window lists functions imported from other DLLs. IDA often wraps them in thunk functions (a single `jmp [IAT_entry]`). Calls to the thunk are calls to the imported function.

23.6 Function Boundaries

IDA may fail to identify a function boundary correctly, especially in obfuscated or hand-written code. Signs: code displayed in gray (not in a function), unexpected `retn` instructions, or missing function prologue. Use P to create a function at the current address.

23.7 Switch Tables

IDA represents switch statements as a jump through a table:

```

jmp ds:off_401234[eax*4]    ; jump table indexed by EAX

```

IDA annotates the cases. If it does not detect the switch, the jump target appears as an indirect jump with no context.

23.8 Thunk Functions

A thunk is a minimal function that only jumps to another:

```
_VirtualProtect:
    jmp ds:[__imp__VirtualProtect@16]
```

Calls to thunks are effectively calls to the target. IDA usually resolves the name.

23.9 Type Propagation

IDA propagates types from known function signatures to their arguments. If IDA knows `recv(SOCKET, char*, int, int)`, it labels the arguments at the call site. This is helpful but can be wrong if the function prototype is misidentified.

23.10 Debugger vs. Disassembler Disagreement

The disassembly shows the on-disk layout. The debugger shows runtime state. Differences arise from:

- Self-modifying code
- Runtime unpacking
- Loader relocations
- Different module base (ASLR vs. IDA's assumed base)

Rebase the IDA database to match the runtime base: Edit -> Segments -> Rebase program.

24. WinDbg Reading Guide

24.1 Register Commands

```
r                ; display all registers
r eax            ; display EAX only
r eax=41414141   ; set EAX to 0x41414141
```

24.2 Disassembly

```
u eip            ; disassemble 8 instructions at EIP
u eip L20        ; disassemble 32 instructions at EIP
ub eip          ; disassemble backward from EIP
uf <address>     ; disassemble entire function
u <addr1> <addr2> ; disassemble range
```

24.3 Memory Display

```
dd <addr>        ; display DWORDs (4-byte values)
dd <addr> L8      ; display 8 DWORDs
db <addr>         ; display bytes
db <addr> L40     ; display 64 bytes
dc <addr>         ; display bytes with ASCII
da <addr>         ; display ASCII string
du <addr>         ; display Unicode string
dps <addr>        ; display pointers with symbols
dds <addr>        ; display DWORDs with symbols
```

`dd` shows values as the CPU interprets them (little-endian reconstructed). `db` shows raw byte order in memory.

24.4 Stack Commands

```
k                ; call stack (return addresses)
kb              ; call stack with first 3 args
```

```
kv                ; call stack with calling convention info
kn                ; call stack with frame numbers
```

24.5 Module Commands

```
lm                ; list all loaded modules
lm m kernel32     ; show kernel32 module info
lm m <pattern>    ; wildcard search for modules
```

24.6 Memory Protection

```
!address <addr>   ; full details of memory region
!vprot <addr>     ; page protection of specific address
!address -summary  ; memory usage summary
```

24.7 Structure Display

```
dt ntdll!_PEB @$peb          ; display PEB structure
dt ntdll!_TEB @$teb          ; display TEB structure
dt ntdll!_PEB_LDR_DATA poi($peb+0xc) ; display loader data
dt ntdll!_LDR_DATA_TABLE_ENTRY <addr> ; display module entry
```

24.8 Exception State

```
.exr -1           ; display most recent exception record
.ecxr             ; switch to exception context (registers at fault)
!analyze -v       ; verbose crash analysis
```

24.9 First-Chance vs. Second-Chance Exceptions

A **first-chance exception** is the debugger's first notification. The application has not yet had a chance to handle it. If the debugger passes it (g), the application's exception handlers run. If no handler handles it, the debugger gets a **second-chance exception** – this is the unhandled crash.

For SEH exploitation: break on first-chance (sxe av) to inspect the corrupted SEH chain before dispatch. If you wait for second-chance, you've missed the window.

24.10 Context Distinction

- **Current context:** the thread/frame the debugger is focused on (set by .frame, .thread)
- **Exception context:** the register state at the time of the fault (switch with .ecxr)
- **Current EIP:** where the debugger is stopped (may differ from the faulting instruction after an exception)

After a crash, EIP shown by r may not be the faulting instruction. Use .ecxr to see the actual state at the fault, then u @eip to see the faulting instruction.

24.11 Breakpoints

```
bp <addr>         ; set breakpoint
bl               ; list breakpoints
bc *             ; clear all breakpoints
bp <addr> ".if @eax==0x40 {} .else {gc}" ; conditional breakpoint
ba r4 <addr>      ; hardware breakpoint (read, 4 bytes)
ba w4 <addr>      ; hardware breakpoint (write, 4 bytes)
ba e1 <addr>      ; hardware breakpoint (execute, 1 byte)
```

24.12 Exception Handling Configuration

```
sxe av          ; break on first-chance access violation
sxd av          ; pass first-chance AV to application
sxe ld <module> ; break when module is loaded
```

24.13 Useful Expressions

```
? 0x40 + 4      ; evaluate arithmetic
? 0 - 0x1c      ; compute negative (two's complement)
? poi(esp)      ; dereference ESP
? @eax + @ecx   ; register arithmetic
```

25. Python Exploit-Development Equivalents

25.1 Packing and Unpacking

```
from struct import pack, unpack

def p32(v: int) -> bytes:
    """Pack a 32-bit integer as little-endian bytes."""
    return pack("<I", v & 0xFFFFFFFF)

def u32(data: bytes) -> int:
    """Unpack 4 little-endian bytes to an integer."""
    return unpack("<I", data[:4])[0]

# Examples
p32(0x625011AF)      # b'\xaf\x11Pb'
u32(b'\xaf\x11Pb')   # 0x625011AF
```

25.2 Byte Patterns

```
# Repeated bytes
padding = b"A" * 68          # 68-byte fill
nop_sled = b"\x90" * 16      # 16 NOPs

# Building a payload
buf = b""
buf += b"A" * 68              # fill to offset
buf += p32(0x625011AF)        # overwrite EIP
buf += b"\x90" * 16           # NOP sled
buf += shellcode              # payload
```

25.3 ROP Chain Construction

```
rop = b""
rop += p32(0x10015442)        # pop eax; ret
rop += p32(0xFFFFFFFF)        # value for EAX (-1)
rop += p32(0x10012345)        # neg eax; ret (EAX = 1)
rop += p32(0x10019876)        # pop ecx; ret
rop += p32(0x00000040)        # value for ECX
rop += p32(0x1001ABCD)        # mov [eax], ecx; ret
```

25.4 Bad Character Checking

```
def check_badchars(data: bytes, bad: bytes = b"\x00") -> list[int]:
    """Return offsets of bad characters found in data."""
    return [i for i, b in enumerate(data) if b in bad]

bad_chars = b"\x00\x0a\x0d\x20"
offsets = check_badchars(payload, bad_chars)
if offsets:
    print(f"Bad chars at offsets: {offsets}")
```

25.5 Socket Communication

```
import socket

def send_payload(host: str, port: int, payload: bytes) -> None:
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((host, port))
        s.send(payload)
        response = s.recv(1024)
```

25.6 Address Formatting

```
# Display an address
addr = 0x625011AF
print(f"0x{addr:08X}")          # "0x625011AF"

# Format as byte string for display
data = p32(addr)
print(data.hex())               # "af115062"
print(" ".join(f"{b:02x}" for b in data)) # "af 11 50 62"
```

25.7 Offset Calculation

```
# Calculate two's complement negative
def neg32(v: int) -> int:
    return (~v + 1) & 0xFFFFFFFF

neg32(0x1C)                    # 0xFFFFFEE4

# Split a value into null-free components
target = 0x00000040
a = 0x80808080
b = (target - a) & 0xFFFFFFFF # 0x7F7F7FC0
assert (a + b) & 0xFFFFFFFF == target
```

25.8 ROR-13 Hash Computation

```
def ror13_hash(name: bytes) -> int:
    h = 0
    for b in name:
        h = ((h >> 13) | (h << 19)) & 0xFFFFFFFF
        h = (h + b) & 0xFFFFFFFF
    return h
```

26. Common Analyst Mistakes

1. **Confusing an address with the data at that address.** `EAX = 0x0012FF80` means EAX holds the number 0x0012FF80. `[EAX]` is the data stored at that address. These are not the same thing.
2. **Forgetting little-endian storage.** The address 0x625011AF is stored as bytes AF 11 50 62. Forgetting this when reading db output or constructing payloads produces wrong addresses.
3. **Assuming EBP is always a frame pointer.** Optimized code uses EBP as a general register (FPO). Treating EBP-relative offsets as frame references in FPO code produces nonsense.
4. **Reading pseudocode without checking assembly.** IDA's decompiler is a useful approximation. It can misrepresent casts, loop boundaries, and calling conventions. Verify against the actual instructions.
5. **Forgetting that CALL pushes a return address.** `call target` decrements ESP by 4 and writes the return address before transferring control. The callee's ESP is 4 less than the caller's ESP at the call site.
6. **Forgetting that RET consumes 4 bytes.** `ret` pops EIP from `[ESP]` and increments ESP by 4. This stack movement must be accounted for in ROP chain layout.
7. **Misunderstanding RET N.** `ret 0x10` does NOT pop 0x10 bytes before reading EIP. It pops EIP first (`ESP += 4`), then adds 0x10 to ESP (`ESP += 0x10`). Total: `ESP += 0x14`. The 0x10 cleans the arguments, not the return address.
8. **Treating LEA as a dereference.** `lea eax, [ebx+4]` computes `EBX+4` and stores it in EAX. It does NOT read memory. `mov eax, [ebx+4]` reads memory.
9. **Confusing signed and unsigned branches.** `ja` is unsigned above; `jg` is signed greater. After `cmp eax, 0x80`, they behave differently if EAX contains a value with the high bit set.
10. **Ignoring gadget side effects.** A gadget like `pop eax; pop ecx; ret` consumes 12 bytes of stack (two values + return address), not 8. The extra pop clobbers a register and consumes a stack slot.
11. **Forgetting stack movement caused by POP.** Each `pop` advances ESP by 4. A gadget with three pops before `ret` consumes 16 bytes of stack total (12 for pops + 4 for ret).
12. **Assuming PUSHAD pushes in memory order.** `PUSHAD` pushes EAX first (deepest) and EDI last (on top). The CPU push order and the resulting memory layout are inverted because the stack grows downward.
13. **Forgetting writable storage for VirtualProtect.** The `lpfl0ldProtect` parameter must point to writable memory. If it points to non-writable memory, `VirtualProtect` fails silently. A writable .data section address or stack address works.
14. **Using an RVA as a VA.** An RVA is an offset from the module base. Using it as an absolute address accesses the wrong memory. Always compute `VA = base + RVA`.
15. **Assuming a module is non-ASLR without checking.** Verify with `!nmod` or by checking the PE header's `DllCharacteristics` for the `IMAGE_DLLCHARACTERISTICS_DYNAMIC_BASE` flag (0x0040).
16. **Assuming all stack memory is currently committed.** The full stack reservation (typically 1 MB) is virtual address space. Only a portion is committed. Guard pages extend the commit on demand. Writing far below ESP without probing can cause a stack overflow exception.
17. **Confusing stack reservation with active stack frames.** A thread's stack size is not determined by the number of functions in the executable. It is a fixed reservation. Active frames are created dynamically as functions are called and destroyed as they return.
18. **Assuming a function knows how its arguments reached the stack.** A `stdcall` function executes `ret N` regardless of whether it was reached by `call`, a ROP gadget, or an overwritten return address. The CPU follows the ABI mechanically.

27. Fast-Reference Tables

27.1 Register Summary

Register	Size	Typical role	Caller/Callee saved
EAX	32	Return value, accumulator	Caller
EBX	32	General purpose, base	Callee
ECX	32	Counter, fastcall arg1, this	Caller
EDX	32	Data, mul/div high, fastcall arg2	Caller
ESI	32	Source index, string ops	Callee
EDI	32	Dest index, string ops	Callee
ESP	32	Stack pointer	—
EBP	32	Frame pointer (convention)	Callee

27.2 Flags Summary

Flag	Bit	Meaning
CF	0	Carry / unsigned overflow
PF	2	Parity of low byte
ZF	6	Result is zero
SF	7	Result is negative (MSB set)
OF	11	Signed overflow
DF	10	String direction (0=fwd, 1=bwd)

27.3 Operand Sizes

Qualifier	Size	Register examples
BYTE PTR	8 bit	AL, BL, CL, DL, AH, BH, CH, DH
WORD PTR	16 bit	AX, BX, CX, DX, SI, DI, SP, BP
DWORD PTR	32 bit	EAX, EBX, ECX, EDX, ESI, EDI, ESP, EBP

27.4 Conditional Jump Quick Reference

Unsigned	Signed	Condition
JA	JG	>
JAEC	JGE	>=
JB	JL	<
JBEC	JLE	<=
JE	JE	==
JNE	JNE	!=

27.5 Calling Conventions

Convention	Args	Cleanup	First reg args	Suffix example
cdecl	R-to-L stack	Caller	None	add esp, N after call
stdcall	R-to-L stack	Callee	None	ret N
fastcall	R-to-L stack	Callee	ECX, EDX	ret N (N excludes reg args)
thiscall	R-to-L stack	Callee	ECX (this)	ret N

27.6 Stack Effects

Instruction	ESP change
push reg	ESP -= 4
pop reg	ESP += 4
call target	ESP -= 4
ret	ESP += 4
ret N	ESP += 4 + N
pushad	ESP -= 32
popad	ESP += 32
pushfd	ESP -= 4
popfd	ESP += 4
sub esp, N	ESP -= N
add esp, N	ESP += N
enter N, 0	ESP -= (4 + N)
leave	ESP = EBP + 4

27.7 Common Gadget Effects

Gadget	Stack consumed	Registers changed
pop eax; ret	8	EAX
pop eax; pop ecx; ret	12	EAX, ECX
xchg eax, esp; ret	4 (at new ESP)	EAX, ESP
mov [edi], eax; ret	4	memory at [EDI]
add esp, 0x20; ret	0x24	ESP
pushad; ret	-28 (net: push 32, pop 4)	ESP
mov eax, [eax]; ret	4	EAX
neg eax; ret	4	EAX, flags
inc eax; ret	4	EAX, flags

27.8 Page Protection Constants

Constant	Value
PAGE_NOACCESS	0x01
PAGE_READONLY	0x02
PAGE_READWRITE	0x04
PAGE_EXECUTE	0x10
PAGE_EXECUTE_READ	0x20
PAGE_EXECUTE_READWRITE	0x40
PAGE_EXECUTE_WRITECOPY	0x80

27.9 Allocation Type Constants

Constant	Value
MEM_COMMIT	0x1000
MEM_RESERVE	0x2000
MEM_COMMIT MEM_RESERVE	0x3000

27.10 PE Export Resolution Offsets

Field	Location
e_lfanew	DllBase + 0x3C
Export Dir RVA	DllBase + e_lfanew + 0x78
NumberOfNames	ExportDir + 0x18
AddressOfFunctions	ExportDir + 0x1C
AddressOfNames	ExportDir + 0x20
AddressOfNameOrdinals	ExportDir + 0x24

27.11 PEB Walking Offsets

Step	Offset
TEB -> PEB	FS:[0x30]
PEB -> Ldr	PEB + 0x0C
Ldr -> InLoadOrderModuleList	Ldr + 0x0C
Ldr -> InMemoryOrderModuleList	Ldr + 0x14
Ldr -> InInitializationOrderModuleList	Ldr + 0x1C
Entry -> DllBase	Entry + 0x18
Entry -> BaseDllName	Entry + 0x2C

27.12 SEH Chain Offsets

Field	Location
SEH chain head	FS:[0x00]
Next record	[record + 0x00]
Handler	[record + 0x04]
Chain terminator	Next = 0xFFFFFFFF

27.13 Common WinDbg Commands

Command	Purpose
r	Show registers
u <addr>	Disassemble
ub <addr>	Disassemble backward
uf <addr>	Disassemble function
dd <addr>	Display DWORDs
db <addr>	Display bytes
dc <addr>	Bytes + ASCII
da <addr>	Display ASCII string
du <addr>	Display Unicode string
dps <addr>	Pointers with symbols
dds <addr>	DWORDs with symbols
k / kb / kv	Call stack
lm	List modules
!address <addr>	Memory region info
!vprot <addr>	Page protection
!teb	Thread environment block
!peb	Process environment block
dt <type> <addr>	Display structure
.exr -1	Last exception record
.ecxr	Exception context
!analyze -v	Verbose crash analysis

Command	Purpose
bp <addr>	Set breakpoint
ba r4 <addr>	Hardware read breakpoint
ba w4 <addr>	Hardware write breakpoint
sxe av	Break on access violation
? <expr>	Evaluate expression

27.14 Python Packing Formats

Format	Type	Size	Endian
<I	uint32	4	Little
<i	int32	4	Little
<H	uint16	2	Little
<h	int16	2	Little
<B	uint8	1	–
<b	int8	1	–
>I	uint32	4	Big

28. Worked State-Transition Exercises

Work through each exercise by tracking register and memory state instruction by instruction. Answers follow each problem.

28.1 Exercise 1: POP and XCHG

Initial state:

EAX = 0x12345678
ECX = 0xAAAAAAAA

ESP -> 0x11111111
0x22222222
0x33333333

Instructions:

```
pop ecx
xchg eax, ecx
ret
```

Question: What are EAX, ECX, EIP, and ESP after execution?

Answer:

After pop ecx:
ECX = 0x11111111
ESP -> 0x22222222
0x33333333

After xchg eax, ecx:
EAX = 0x11111111
ECX = 0x12345678
ESP -> 0x22222222
0x33333333

After ret:
EIP = 0x22222222
ESP -> 0x33333333
EAX = 0x11111111
ECX = 0x12345678

28.2 Exercise 2: CALL and RET

Initial state:

EIP = 0x00401000 (address of the CALL instruction)
ESP = 0x0012FF80

Instruction at 0x00401000: call 0x00402000 (5 bytes long)
Instruction at 0x00402000: ret

Question: What are EIP and ESP after the CALL executes? After the RET?

Answer:

After call 0x00402000:
EIP = 0x00402000
ESP = 0x0012FF7C
[0x0012FF7C] = 0x00401005 (return address = next instruction after CALL)

After ret:
EIP = 0x00401005
ESP = 0x0012FF80

28.3 Exercise 3: RET N

Initial state:

ESP = 0x0012FF80
[0x0012FF80] = 0x00401000 (return address)
[0x0012FF84] = 0x00000001 (arg1)
[0x0012FF88] = 0x00000002 (arg2)
[0x0012FF8C] = 0x00000003 (arg3)
[0x0012FF90] = 0xDEADBEEF (caller's data)

Instruction: ret 0x0C

Question: What are EIP and ESP after execution?

Answer:

EIP = 0x00401000 (popped from [0x0012FF80])
ESP = 0x0012FF80 + 4 + 0x0C = 0x0012FF90
 ^ret ^cleanup 3 args
Next value at [ESP] = 0xDEADBEEF

28.4 Exercise 4: PUSH and POP Sequence

Initial state:

EAX = 0xAAAAAAAA
EBX = 0BBBBBBBB
ESP = 0x0012FF80

Instructions:

```
push eax
push ebx
pop  eax
pop  ebx
```

Question: What are EAX, EBX, and ESP after execution?

Answer:

```
After push eax:  ESP=0x0012FF7C  [0x0012FF7C]=0xAAAAAAAA
After push ebx:  ESP=0x0012FF78  [0x0012FF78]=0xBBBBBBBB
After pop  eax:  ESP=0x0012FF7C  EAX=0xBBBBBBBB  (was EBX's value)
After pop  ebx:  ESP=0x0012FF80  EBX=0xAAAAAAAA  (was EAX's value)
```

Final: EAX=0xBBBBBBBB, EBX=0xAAAAAAAA (swapped), ESP=0x0012FF80

28.5 Exercise 5: LEAVE

Initial state:

```
EBP = 0x0012FF80
ESP = 0x0012FF40
[0x0012FF80] = 0x0012FFA0  (saved EBP from caller)
[0x0012FF84] = 0x00401000  (return address)
```

Instructions:

```
leave
ret
```

Question: What are EBP, ESP, and EIP after execution?

Answer:

```
leave = mov esp, ebp; pop ebp:
  mov esp, ebp:  ESP = 0x0012FF80
  pop ebp:      EBP = [0x0012FF80] = 0x0012FFA0
                  ESP = 0x0012FF84
```

```
ret:
  EIP = [0x0012FF84] = 0x00401000
  ESP = 0x0012FF88
```

Final: EBP=0x0012FFA0, ESP=0x0012FF88, EIP=0x00401000

28.6 Exercise 6: Pointer Dereference Chain

Initial state:

```
EAX = 0x00300000
```

Memory:

```
[0x00300000] = 0x00400000
[0x00400000] = 0x00500000
[0x00500000] = 0x41414141
```

Instructions:

```
mov eax, [eax]
mov eax, [eax]
mov eax, [eax]
```

Question: What is EAX after each instruction?

Answer:

After 1st `mov eax, [eax]`: EAX = [0x00300000] = 0x00400000

After 2nd `mov eax, [eax]`: EAX = [0x00400000] = 0x00500000

After 3rd `mov eax, [eax]`: EAX = [0x00500000] = 0x41414141

Three-level pointer chain. This is how PEB walking works (follow pointer, follow pointer, read value).

28.7 Exercise 7: LEA vs. MOV

Initial state:

EBX = 0x0012FF80

ECX = 0x00000003

Memory:

[0x0012FF8C] = 0xDEADBEEF

Instructions (independent, not sequential):

```
; A:
mov eax, [ebx+ecx*4]

; B:
lea eax, [ebx+ecx*4]
```

Question: What is EAX after instruction A? After instruction B?

Answer:

Effective address = 0x0012FF80 + 3*4 = 0x0012FF80 + 0x0C = 0x0012FF8C

A (`mov`): EAX = [0x0012FF8C] = 0xDEADBEEF (reads memory)

B (`lea`): EAX = 0x0012FF8C (computes address only)

28.8 Exercise 8: Stack Pivot

Initial state:

EAX = 0x0C0C0C0C

ESP = 0x0012FF80

Memory (the fake ROP chain at 0x0C0C0C0C):

[0x0C0C0C0C] = 0x10015442 (pop ecx; ret)

[0x0C0C0C10] = 0x00000040 (value for ECX)

[0x0C0C0C14] = 0x10019876 (next gadget address)

Instructions:

```
xchg eax, esp
ret
```

Question: Trace the state through the pivot and the first gadget.

Answer:

```
xchg eax, esp:
  EAX = 0x0012FF80 (old ESP)
  ESP = 0x0C0C0C0C (old EAX = pivot target)
```

```
ret:
  EIP = [0x0C0C0C0C] = 0x10015442
  ESP = 0x0C0C0C10
```

Now executing "pop ecx; ret" at 0x10015442:

```
pop ecx:
  ECX = [0x0C0C0C10] = 0x00000040
  ESP = 0x0C0C0C14
ret:
  EIP = [0x0C0C0C14] = 0x10019876
  ESP = 0x0C0C0C18
```

Execution continues at the next gadget with ECX = 0x40.

28.9 Exercise 9: VirtualProtect ROP Frame

Initial state (after ROP chain arranges the stack):

```
ESP -> 0x7C801234 (VirtualProtect address)
+04 0x0C0C0C40 (return address = shellcode)
+08 0x0C0C0C40 (arg1: lpAddress = shellcode)
+0C 0x00000400 (arg2: dwSize = 1024)
+10 0x00000400 (arg3: flNewProtect = PAGE_EXECUTE_READWRITE)
+14 0x10004000 (arg4: lpflOldProtect = writable .data address)
```

Instruction: ret (from the preceding gadget)

Question: What does VirtualProtect see? Where does execution go after VirtualProtect returns?

Answer:

```
ret:
  EIP = 0x7C801234 (VirtualProtect)
  ESP = old_ESP + 4 (now points at 0x0C0C0C40)
```

VirtualProtect sees (stdcall frame):

```
[ESP+0x00] = 0x0C0C0C40 (return address)
[ESP+0x04] = 0x0C0C0C40 (lpAddress)
[ESP+0x08] = 0x00000400 (dwSize)
[ESP+0x0C] = 0x00000400 (flNewProtect)
[ESP+0x10] = 0x10004000 (lpflOldProtect)
```

VirtualProtect executes:

- Changes page containing 0x0C0C0C40 to RWX
- Writes old protection to [0x10004000]
- Executes ret 0x10:


```
EIP = 0x0C0C0C40 (shellcode)
ESP = ESP + 4 + 0x10 (cleaned 4 args)
```

Execution begins at shellcode. The page is now executable.

28.10 Exercise 10: PUSHAD

Initial state:

```
EAX = 0x11111111    EDI = 0x7C801234
ECX = 0x22222222    ESI = 0x0C0C0C40
EDX = 0x33333333    EBP = 0x0C0C0C40
EBX = 0x00000040    ESP = 0x0012FF80
```

Instructions:

```
pushad
ret
```

Question: What does the stack look like after PUSHAD? Where does RET go?

Answer:

PUSHAD pushes in order: EAX, ECX, EDX, EBX, ESP(orig), EBP, ESI, EDI
 ESP decreases by 32: $0x0012FF80 - 0x20 = 0x0012FF60$

Stack after PUSHAD:

```
[0x0012FF60] = 0x7C801234 (EDI) <-- ESP points here
[0x0012FF64] = 0x0C0C0C40 (ESI)
[0x0012FF68] = 0x0C0C0C40 (EBP)
[0x0012FF6C] = 0x0012FF80 (original ESP)
[0x0012FF70] = 0x00000040 (EBX)
[0x0012FF74] = 0x33333333 (EDX)
[0x0012FF78] = 0x22222222 (ECX)
[0x0012FF7C] = 0x11111111 (EAX)
```

ret:

```
EIP = [0x0012FF60] = 0x7C801234 (= EDI = VirtualProtect)
ESP = 0x0012FF64
```

VirtualProtect now sees:

```
[ESP+0x00] = 0x0C0C0C40 (ESI = return addr / shellcode)
[ESP+0x04] = 0x0C0C0C40 (EBP = lpAddress)
[ESP+0x08] = 0x0012FF80 (orig ESP = dwSize -- large enough)
[ESP+0x0C] = 0x00000040 (EBX = flNewProtect = PAGE_EXECUTE_READWRITE)
[ESP+0x10] = 0x33333333 (EDX = lpflOldProtect -- must be writable!)
```

Note: EDX (0x33333333) must point to writable memory or the call fails.

28.11 Exercise 11: PEB Walking

Initial state:

```
FS:[0x30] = 0x7FFD3000 (PEB)
```

Memory:

```
[0x7FFD3000 + 0x0C] = 0x77F40000 (Ldr)
[0x77F40000 + 0x14] = 0x77F40120 (InMemoryOrderModuleList.Flink)
```

At the first entry (ESI = 0x77F40120):

```
[0x77F40120 + 0x00] = 0x77F40130 (Flink to next)
[0x77F40120 + 0x10] = 0x00400000 (DllBase of exe)
```

At the second entry (ESI = 0x77F40130):

```
[0x77F40130 + 0x00] = 0x77F40140 (Flink to next)
```

`[0x77F40130 + 0x10] = 0x7C900000` (DllBase of ntdll.dll)

At the third entry (`ESI = 0x77F40140`):

`[0x77F40140 + 0x00] = 0x77F40014` (Flink = list head)

`[0x77F40140 + 0x10] = 0x7C800000` (DllBase of kernel32.dll)

Instructions:

```
xor ecx, ecx
mov eax, fs:[ecx+0x30]    ; EAX = PEB
mov eax, [eax+0x0C]      ; EAX = Ldr
mov esi, [eax+0x14]      ; ESI = first Flink
lodsd                   ; EAX = [ESI], ESI += 4 (follow Flink once)
                        ; -- but this reads the Flink, moving to entry 2
```

Question: After `lodsd`, what is `EAX`? How do you reach `kernel32.dll`'s base?

Answer:

After `lodsd`:

`EAX = [0x77F40120] = 0x77F40130` (Flink to second entry)

`ESI = 0x77F40124`

To reach `kernel32` (third entry), follow Flink again:

`mov esi, eax` ; `ESI = 0x77F40130` (second entry)

`lodsd` ; `EAX = [0x77F40130] = 0x77F40140` (third entry)

Now read DllBase:

`mov ebx, [eax+0x10]` ; `EBX = [0x77F40140+0x10] = 0x7C800000 = kernel32 base`

The `InMemoryOrderLinks` field is at offset `+0x08` in the entry structure. Since we entered the list through `Ldr+0x14` (which points to the `Links` field), the DllBase (at `entry+0x18`) is at `[Flink+0x10]` (`0x18 - 0x08 = 0x10`).